

第 6 卷 数学与字符串模板

考试速查：主查常用数学与字符串：gcd/lcm、模运算、快速幂、逆元、组合数、筛法、质因数分解、方程组求解、一元方程求根、KMP/Z、Trie、Hash 和 Manacher。

Contents

第 6 卷 数学与字符串模板	2
考试速查定位	2
数学与字符串速查索引	2
MATH-01 gcd / lcm	2
MATH-02 快速幂与模运算	5
MATH-03 逆元与组合数预处理	7
MATH-04 筛法与质因数分解	10
MATH-05 矩阵快速幂	13
MATH-06 容斥基础	16
STR-01 string 常用操作	19
STR-02 KMP 与 Z 函数	21
STR-03 Trie 与 Rolling Hash	24
STR-04 AC 自动机 (低优先级)	27
STR-05 Manacher 回文算法	32
SIM-01 模拟、字符串扫描与高精度	37
SIM-02 手写高精度 BigInteger 类	42
模型选择卡	43
SIM-03 表达式求值与 AST 语法树	51
为什么 AST 比直接求值更通用	52
SIM-04 JSON / CSV / INI 解析器	58
模块选择卡	59
SIM-05 手写编译器 / 解释器骨架	68
三层结构	69
本模板支持的语法	69
SIM-06 日期、时间、时区与历法	77
最重要思想：日期先转整数	78
夏令时怎么处理	82
SIM-07 方程求解：高斯消元、模方程、一元方程与多项式	84
先判断题目属于哪一类	85
高斯消元的行列约定	85
二分和牛顿怎么选	92
多项式求根的低心智路线	92

第 6 卷 数学与字符串模板

考试速查定位

第 6 卷 数学与字符串模板

- **这是第几卷：** 06。
- **主要查什么：** 主查常用数学与字符串：gcd/lcm、模运算、快速幂、逆元、组合数、筛法、质因数分解、方程组求解、一元方程求根、KMP/Z、Trie、Hash 和 Manacher。
- **什么时候翻：** 快速幂、组合数、筛法、方程求解、KMP、Trie、Hash、字符串 DP 时翻。
- **题面关键词：** gcd/lcm；快速幂；逆元；组合数；筛；质因数；高斯消元；二分求根；KMP；Z；Trie；Hash；Manacher。

自动由 MATH/STR/SIM 模块重建。定位是常用数论、组合、矩阵、方程求解、字符串和模拟补充。

数学与字符串速查索引

题面信号	模块入口
gcd/lcm/快速幂/逆元	MATH-01/02/03
组合数/计数/阶乘	MATH-03/04
线性递推/矩阵快速幂	MATH-05
质数/因数/筛法	MATH-06
大数/高精度/BigInteger	SIM-01/02
表达式求值/AST	SIM-03
JSON/CSV/INI 解析	SIM-04
手写解释器/小语言模拟	SIM-05
日期/时区/历法	SIM-06
方程组/高斯消元/求根	SIM-07
字符串匹配/前缀/哈希	STR-02/03
最长回文/回文半径/区间回文判断	STR-05 Manacher

MATH-01 gcd / lcm

模块编号：MATH-01

模块名称：最大公约数与最小公倍数

标签：[数学][数论][gcd][lcm]

一句话用途：快速处理整除、约分、周期同步、比例化简和两个数的公共因子问题。

题面触发词：

- 最大公约数、最小公倍数。
- 互质、约分、最简分数。
- 周期同时发生、两个循环何时重合。
- 能否整除、公共因子、最大公共长度。
- 把若干数按比例化简。

什么时候用：

- 题目明显问 $\gcd(a,b)$ 、 $\text{lcm}(a,b)$ 。
- 需要判断 $\gcd(a,b)=1$ 。
- 需要把分数 x/y 约成最简。
- 周期题中要求两个周期第一次同时出现，常用 lcm 。

不要什么时候用：

- 需要所有因子列表时，只求 gcd 不够，要枚举因子或质因数分解。
- lcm 可能超过 long long 时不能直接乘。
- 浮点数比例不要直接 gcd，先转成整数或避免浮点。

复杂度：

- $\gcd(a,b)$: $O(\log \min(a,b))$ 。
- 多个数的 gcd/lcm：每加入一个数做一次 gcd。

数据范围参考：

- $a, b \leq 1e18$: long long 可存, lcm 乘法要防溢出。
- $n \leq 2e5$: 顺序合并 gcd/lcm 可用。

依赖的标准容器:

- 不依赖特殊容器。
- 多个数时使用 1-index 数组 `vector<ll> a(n + 1)`。

输入如何整理:

```
int n;
cin >> n;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口:

`gcd_ll(a,b)` -> 最大公约数

`lcm_ll(a,b)` -> 最小公倍数, 普通版

`lcm_limit(a,b,limit)` -> 超过 limit 时返回 limit + 1; 若 limit 已经是 LLONG_MAX, 则返回 limit

输出能力:

- 两数或多数组组合的最大公约数。
- 两数或多数组组合的最小公倍数。
- 互质判断。
- 分数约分。

下游可接:

- 逆元和模运算里的互质判断。
- 中国剩余类问题的前置检查。
- 周期 DP、模拟题、字符串周期题。

可拼接模块:

- MATH-02 模运算。
- MATH-03 逆元。
- MATH-04 质因数分解。
- STR-02 KMP/Z 函数求字符串周期后接 gcd/lcm。

模数是否为质数的分支:

本模块本身不依赖模数。

如果后续要在 mod 下做除法:

mod 是质数 -> 可接费马逆元。

mod 不是质数 -> 必须检查 `gcd(x, mod) == 1`, 再用扩展 gcd 求逆元。

模板代码:

```
using ll = long long;

ll gcd_ll(ll a, ll b) {
    if (a == LLONG_MIN || b == LLONG_MIN) {
        // 极端数据兜底; 普通竞赛题很少卡 LLONG_MIN。
        unsigned long long x = a < 0 ? 0ULL - (unsigned long long)a : (unsigned long long)a;
        unsigned long long y = b < 0 ? 0ULL - (unsigned long long)b : (unsigned long long)b;
        while (y) {
            unsigned long long t = x % y;
            x = y;
            y = t;
        }
        return (ll)x;
    }
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) {
        ll t = a % b;
        a = b;
        b = t;
    }
}
```

```

    return a;
}

ll lcm_ll(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    ll g = gcd_ll(a, b);
    __int128 x = (__int128)a / g * b;
    if (x < 0) x = -x;
    assert(x <= LLONG_MAX); // 若可能超过 Long Long, 改用 lcm_limit.
    return (ll)x;
}

ll lcm_limit(ll a, ll b, ll limit) {
    if (a == 0 || b == 0) return 0;
    __int128 aa = a, bb = b;
    if (aa < 0) aa = -aa;
    if (bb < 0) bb = -bb;
    ll g = gcd_ll(a, b);
    aa /= g;
    __int128 lim = limit;
    ll over = (limit == LLONG_MAX ? limit : limit + 1);
    if (bb != 0 && aa > lim / bb) return over;
    __int128 res = aa * bb;
    if (res > lim) return over;
    return (ll)res;
}

```

调用示例:

```

ll g = 0;
for (int i = 1; i <= n; i++) g = gcd_ll(g, a[i]);

ll l = 1;
for (int i = 1; i <= n; i++) {
    l = lcm_limit(l, a[i], 4'000'000'000'000'000'000LL);
}

if (gcd_ll(x, y) == 1) {
    // x 和 y 互质
}

```

常见坑:

- $\text{lcm} = a * b / \text{gcd}(a, b)$ 容易先乘溢出, 写成 $a / \text{gcd} * b$ 。
- $\text{gcd}(0, x) = \text{abs}(x)$, 多数组合时初始值可设为 0。
- 负数取 gcd 时先转正; 如果题目可能出现 LLONG_MIN, 用本页安全版, 不要直接 $a = -a$ 。
- C++17 有 `std::gcd`, 但纸质模板自己写更可控。
- `lcm_ll` 只在结果保证不超过 long long 时使用; 可能爆时用 `lcm_limit`。

暴力/部分分替代:

- 小数据可以从 $\min(a, b)$ 往下枚举第一个同时整除的数求 gcd。
- 小数据可以从 $\max(a, b)$ 往上枚举第一个同时被整除的数求 lcm。
- 周期题不会推公式时, 可先模拟到 `lcm_limit` 的上限拿部分分。

升级方向:

- 多次区间 gcd 查询 -> Sparse Table 或 Segment Tree。
- 需要因子个数/因子和 -> 质因数分解。
- 模意义下除法 -> 逆元。

最小测试样例:

```

gcd_ll(12, 18) = 6
lcm_ll(12, 18) = 36
gcd_ll(0, 5) = 5
lcm_limit(1000000000000, 1000000000000, 1000000000000) = 1000000000000

```

MATH-02 快速幂与模运算

模块编号: MATH-02

模块名称: 快速幂、取模规范与乘法防溢出

标签: [数学][快速幂][模运算][二进制拆分]

一句话用途: 在 $O(\log b)$ 内计算 a^b , 并把所有加减乘幂统一到安全的取模写法。

题面触发词:

- 答案对 $1e9+7$ / 998244353 取模。
- 求 $a^b \bmod p$ 。
- 指数很大、幂次、翻倍、二进制拆分。
- 方案数很大, 请输出取模结果。
- 乘法可能溢出。

什么时候用:

- 指数 b 可达 $1e9/1e18$, 不能循环乘。
- 计数题要求取模。
- 矩阵快速幂、组合数、逆元都要调用快速幂。
- 中间乘法可能超过 `long long`, 需要 `__int128` 辅助。

不要什么时候用:

- 指数很小且只算一次时, 普通循环也可拿部分分。
- 需要浮点幂时用 `pow`, 但竞赛整数取模不要用浮点 `pow`。
- 模数为 1 时所有结果都是 0, 要提前处理。

复杂度:

- 快速幂: $O(\log b)$ 。
- 单次加减乘取模: $O(1)$ 。

数据范围参考:

- $b \leq 1e18$: `long long` 指数可用。
- $a, \text{mod} \leq 1e18$ 且乘法会溢出: 使用 `mul_mod` 的 `__int128` 版本。

依赖的标准容器:

- 不依赖特殊容器。
- 常量 MOD 推荐用 `long long`。

输入如何整理:

```
ll a, b, mod;
cin >> a >> b >> mod;
```

接口:

```
norm(x, mod) -> 把 x 规范到 [0, mod-1]
add_mod(a,b,mod) -> (a+b)%mod
sub_mod(a,b,mod) -> (a-b)%mod
mul_mod(a,b,mod) -> (a*b)%mod, 防溢出版
pow_mod(a,b,mod) -> a^b % mod
```

输出能力:

- 整数幂取模。
- 安全加减乘取模。
- 作为逆元、组合数、矩阵快速幂的底层函数。

下游可接:

- MATH-03 逆元与组合数。
- MATH-05 矩阵快速幂。
- DP 方案数取模。

可拼接模块:

- 计数 DP + add_mod。
- 组合数预处理 + pow_mod。
- Rolling Hash 预处理幂数组。

模数是否为质数的分支：

只算 $a^b \bmod \text{mod}$ ：mod 是否为质数都可以。

用 $\text{pow_mod}(a, \text{mod}-2, \text{mod})$ 求逆元：只有 mod 是质数且 $a \bmod \text{mod} \neq 0$ 才能这样做。

mod 不是质数：不要套费马逆元；除法要改用扩展 gcd 逆元并检查互质。

模板代码：

```
using ll = long long;

ll norm(ll x, ll mod) {
    assert(mod > 0);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

ll add_mod(ll a, ll b, ll mod) {
    assert(mod > 0);
    return (ll)((__int128)norm(a, mod) + norm(b, mod)) % mod;
}

ll sub_mod(ll a, ll b, ll mod) {
    assert(mod > 0);
    return (ll)((__int128)norm(a, mod) - norm(b, mod) + mod) % mod;
}

ll mul_mod(ll a, ll b, ll mod) {
    assert(mod > 0);
    return (ll)((__int128)norm(a, mod) * norm(b, mod) % mod);
}

ll pow_mod(ll a, ll b, ll mod) {
    assert(mod > 0);
    assert(b >= 0);
    if (mod == 1) return 0;
    a = norm(a, mod);
    ll res = 1 % mod;
    while (b > 0) {
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}
```

调用示例：

```
const ll MOD = 1000000007LL;
cout << pow_mod(2, 10, MOD) << "\n"; // 1024

ll ans = 0;
ans = add_mod(ans, ways, MOD);
ans = sub_mod(ans, bad, MOD);
```

常见坑：

- $(a - b) \bmod \text{mod}$ 在 C++ 里可能是负数，要加 mod。
- $1e9+7$ 是 double 字面量风险，写 1000000007LL。
- $\text{pow}(a, b)$ 返回浮点，不能拿来整数取模。
- $\text{mod} == 1$ 时 $1 \bmod \text{mod}$ 是 0，快速幂也应返回 0。
- long long 乘 long long 可能先溢出，再取模已经晚了。

暴力/部分替代:

- $b \leq 1e6$ 可循环乘 b 次。
- 方案数小数据可先不取模, 用 `long long` 暂存并观察是否溢出。
- 乘法不大时可用 $(a \% \text{mod}) * (b \% \text{mod}) \% \text{mod}$ 。

升级方向:

- 多次幂同底或同模 -> 预处理幂数组。
- 线性递推第 n 项 -> 矩阵快速幂。
- 需要除法 -> 逆元模块。

最小测试样例:

```
pow_mod(2, 10, 1000000007) = 1024
sub_mod(3, 5, 7) = 5
mul_mod(1000000000000000000, 2, 1000000007) = 98
pow_mod(5, 3, 1) = 0
```

MATH-03 逆元与组合数预处理

模块编号: MATH-03

模块名称: 模逆元、阶乘逆元与组合数 $C(n, k)$

标签: [数学][逆元][组合数][阶乘][计数]

一句话用途: 把模意义下的除法变成乘法, 并在 $O(1)$ 查询大量组合数。

题面触发词:

- 组合数、从 n 个选 k 个。
- 排列组合、方案数取模。
- 需要除以某个数后取模。
- $n, q \leq 2e5/1e6$ 且多次询问 $C(n, k)$ 。
- 模数给出为 $1e9+7$ 、 998244353 。

什么时候用:

- 需要计算 $a / b \bmod \text{MOD}$ 。
- 大量查询 $C(n, k) \bmod \text{MOD}$ 。
- 计数 DP 里转移含组合数。
- 排列、路径条数、二项式展开。

不要什么时候用:

- 模数不是质数时, 不要直接用 `pow_mod(x, mod-2, mod)`。
- $k > n$ 时组合数应为 0。
- n 极大但查询很少时, 不一定能预处理到 n , 要考虑 Lucas 或乘法公式。

复杂度:

- 单次费马逆元: $O(\log \text{mod})$ 。
- 阶乘预处理: $O(N)$ 。
- 查询 $C(n, k)$: $O(1)$ 。
- 扩展 gcd 逆元: $O(\log \text{mod})$ 。

数据范围参考:

- $N \leq 2e6$: 阶乘和逆阶乘数组常用。
- $N > 1e7$: 注意内存和预处理时间。
- mod 不是质数: 只能对与 mod 互质的数求逆元。

依赖的标准容器:

- 1-index 或 0-index 阶乘数组都可; 本模板 `fac[0..N]` 是数学自然下标。
- 使用 `vector<ll> fac, ifac`。

输入如何整理:

```
int maxN, q;
ll MOD;
cin >> maxN >> q >> MOD;
Comb comb;
comb.init(maxN, MOD, true); // true 表示确认 MOD 是质数
```

接口:

```
inv_prime(a, mod) -> 质数模数下的逆元
inv_coprime(a, mod) -> 非质数模数下, a 与 mod 互质时的逆元
Comb.init(N, mod, mod_is_prime)
Comb.C(n, k)
Comb.P(n, k)
```

输出能力:

- 模意义下除法。
- 组合数 $C(n, k)$ 。
- 排列数 $P(n, k)$ 。
- 多次查询的快速答案。

下游可接:

- 计数 DP。
- 容斥。
- 组合计数题。
- 概率题的分数取模。

可拼接模块:

- MATH-02 快速幂。
- MATH-06 容斥基础。
- DP 方案数。

模数是否为质数的分支:

mod 是质数:

```
inv(a) = pow_mod(a, mod-2, mod), 要求 a % mod != 0。
fac/ifac 预处理可  $O(N)$ ,  $C(n, k) = fac[n] * ifac[k] * ifac[n-k]$ 。
```

mod 不是质数:

不能用 `pow_mod(a, mod-2, mod)`。
只有 `gcd(a, mod) == 1` 时才有逆元, 用 `exgcd`。
普通 `fac/ifac` 组合数可能失效, 因为 `fac` 里含有与 `mod` 不互质的因子。
考场弱基础策略: 优先找题面是否保证 `mod` 是质数; 没保证时用小数据 Pascal 递推或整数约分法拿部分分。

模板代码:

```
using ll = long long;

ll norm(ll x, ll mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

ll mul_mod(ll a, ll b, ll mod) {
    return (ll)((__int128)norm(a, mod) * norm(b, mod) % mod);
}

ll pow_mod(ll a, ll b, ll mod) {
    assert(b >= 0);
    ll res = 1 % mod;
    a = norm(a, mod);
    while (b > 0) {
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
}
```



```

    return res;
}

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return a >= 0 ? a : -a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    __int128 yy = (__int128)x1 - (__int128)(a / b) * y1;
    assert((__int128)LLONG_MIN <= yy && yy <= (__int128)LLONG_MAX);
    y = (ll)yy;
    return g;
}

ll inv_prime(ll a, ll mod) {
    assert(mod > 1);
    a = norm(a, mod);
    if (a == 0) return -1; // 不存在逆元
    return pow_mod(a, mod - 2, mod);
}

ll inv_coprime(ll a, ll mod) {
    ll x, y;
    ll g = exgcd(a, mod, x, y);
    if (g != 1) return -1; // 不存在逆元
    return norm(x, mod);
}

struct Comb {
    int N;
    ll mod;
    vector<ll> fac, ifac;

    void init(int n, ll mod_, bool mod_is_prime) {
        N = n;
        mod = mod_;
        if (!mod_is_prime || N >= mod) {
            // 阶乘逆元法只适用于质数模且  $N < mod$ ; 非质数模请用 Pascal/扩展组合数。
            N = 0;
            fac.assign(1, 1);
            ifac.assign(1, 1);
            return;
        }
        fac.assign(N + 1, 1);
        ifac.assign(N + 1, 1);
        for (int i = 1; i <= N; i++) fac[i] = mul_mod(fac[i - 1], i, mod);

        ifac[N] = inv_prime(fac[N], mod);

        for (int i = N; i >= 1; i--) ifac[i - 1] = mul_mod(ifac[i], i, mod);
    }

    ll C(int n, int k) {
        if (k < 0 || k > n || n > N) return 0;
        return mul_mod(fac[n], mul_mod(ifac[k], ifac[n - k], mod), mod);
    }

    ll P(int n, int k) {
        if (k < 0 || k > n || n > N) return 0;

```

```

        return mul_mod(fac[n], ifac[n - k], mod);
    }
};

```

调用示例:

```

const ll MOD = 1000000007LL;
Comb comb;
comb.init(1000000, MOD, true);
cout << comb.C(5, 2) << "\n"; // 10

```

```

ll inv2 = inv_prime(2, MOD);
ll half = mul_mod(x, inv2, MOD);

```

常见坑:

- mod 不是质数却套费马逆元, 是最常见错误。
- $a \% \text{mod} == 0$ 没有逆元; 本模板的 `inv_prime` 会返回 -1。
- $C(n, k)$ 中 $k < 0$ 或 $k > n$ 要返回 0。
- 多测时 `init` 要按最大 N 重新建或一次建到全局最大。
- `fac[N]` 在非质数模数下可能不可逆, `ifac` 会失效。
- 阶乘逆元法要求 mod 是质数且 $N < \text{mod}$; 不满足时改 Pascal、Lucas 或扩展组合数。

暴力/部分替代:

- $n \leq 2000$: Pascal 递推 $C[i][j] = C[i-1][j-1] + C[i-1][j]$, 不需要逆元, 非质数模数也能用。
- 查询很少且 k 小: 用乘法公式逐项约分, 或逐项乘后除的整数版。
- 不确定模数是否质数: 先写 Pascal 小范围版本拿部分分。

升级方向:

- n 很大、mod 是小质数 -> Lucas。
- mod 不是质数且 n 大 -> 分解模数/CRT, 难度高, 低优先级。
- 组合数参与容斥 -> 接 MATH-06。

最小测试样例:

```

MOD = 1000000007
inv_prime(2, MOD) = 500000004
C(5,2) = 10
P(5,2) = 20

```

```

MOD = 8
inv_coprime(3, 8) = 3
inv_coprime(2, 8) = -1

```

MATH-04 筛法与质因数分解

模块编号: MATH-04

模块名称: 质数筛、最小质因子与质因数分解

标签: [数学][筛法][质数][质因数分解]

一句话用途: 预处理质数, 快速判断质数、分解整数, 并支持因子个数、因子和、欧拉函数等数论计数。

题面触发词:

- 质数、素数、合数。
- 质因数、分解质因数。
- 因子个数、约数个数、约数和。
- 多次询问某数是否为质数。
- $1..n$ 中有多少质数。

什么时候用:

- 需要对很多数判断质数。
- 需要快速分解许多 $x \leq N$ 的数。
- 需要枚举质数做试除。

- 因子相关计数题。

不要什么时候用：

- 只判断一个很大的数，筛到 $\sqrt{x}$ 可能够，但筛到 x 不行。
- N 达到 $1e9$ 不能开数组筛。
- 只需要 gcd/lcm 时，不一定要分解。

复杂度：

- 埃氏筛： $O(N \log \log N)$ 。
- 线性筛： $O(N)$ 。
- 用最小质因子分解 x ： $O(\text{质因子个数})$ 。
- 用质数表试除分解 x ： $O(\pi(\sqrt{x}))$ 。

数据范围参考：

- $N \leq 1e7$ ：可筛，注意内存。
- 多次分解 $x \leq 1e7$ ：预处理 spf 最稳。
- 单次 $x \leq 1e12$ ：筛到 $1e6$ 后试除。

依赖的标准容器：

- `vector<int> primes`。
- `vector<int> spf`，下标自然从 $0..N$ 。

输入如何整理：

```
int N;
cin >> N;
LinearSieve sieve;
sieve.init(N);
```

接口：

```
LinearSieve.init(N)
is_prime(x)
factorize_by_spf(x)
factorize_by_primes(x, primes)
```

输出能力：

- 质数表。
- 判断 x 是否质数。
- 质因数分解结果 (p, cnt) 。
- 因子个数、因子和的基础材料。

下游可接：

- 组合数非质数模数的高级处理。
- gcd/lcm 分析。
- 容斥中枚举质因子集合。

可拼接模块：

- MATH-01 gcd/lcm。
- MATH-03 逆元中判断互质。
- MATH-06 容斥基础。

模数是否为质数的分支：

本模块可用来判断 mod 是否为质数。

mod 是质数 $\rightarrow$ 逆元/组合数可走费马和 fac/ifac 。

mod 不是质数 $\rightarrow$ 逆元只在 $\text{gcd}(a, \text{mod}) == 1$ 时存在；组合数不要直接 fac/ifac 。

模板代码：

```
using ll = long long;

struct LinearSieve {
    int N;
    vector<int> primes;
    vector<int> spf; // smallest prime factor
```

```

void init(int n) {
    N = n;
    primes.clear();
    spf.assign(N + 1, 0);
    for (int i = 2; i <= N; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > spf[i] || (ll)i * p > N) break;
            spf[i * p] = p;
        }
    }
}

bool is_prime(int x) const {
    if (x < 2 || x > N) return false;
    return spf[x] == x;
}

vector<pair<int, int>> factorize_by_spf(int x) const {
    assert(2 <= x && x <= N);
    vector<pair<int, int>> res;
    while (x > 1) {
        int p = spf[x], cnt = 0;
        while (x % p == 0) {
            x /= p;
            cnt++;
        }
        res.push_back({p, cnt});
    }
    return res;
}

};

vector<pair<ll, int>> factorize_by_primes(ll x, const vector<int> &primes) {
    vector<pair<ll, int>> res;
    if (x <= 1) return res;
    for (int p : primes) {
        if ((ll)p > x / p) break;
        if (x % p == 0) {
            int cnt = 0;
            while (x % p == 0) {
                x /= p;
                cnt++;
            }
            res.push_back({p, cnt});
        }
    }
}

// 如果 primes 没筛到 sqrt(原始 x), 继续朴素试除, 避免把合数静默当质数。
ll start = primes.empty() ? 2LL : (ll)primes.back() + 1;
if (start <= 2 && x % 2 == 0) {
    int cnt = 0;
    while (x % 2 == 0) {
        x /= 2;
        cnt++;
    }
    res.push_back({2, cnt});
    start = 3;
}
if (start % 2 == 0) start++;

```

```

for (ll d = start; d <= x / d; d += 2) {
    if (x % d == 0) {
        int cnt = 0;
        while (x % d == 0) {
            x /= d;
            cnt++;
        }
        res.push_back({d, cnt});
    }
}
if (x > 1) res.push_back({x, 1});
return res;
}

```

调用示例:

```

LinearSieve sieve;
sieve.init(1000000);

```

```

if (sieve.is_prime(97)) {
    // 97 是质数
}

```

```

auto fac = sieve.factorize_by_spf(360);
// fac = (2,3), (3,2), (5,1)

```

```

auto big_fac = factorize_by_primes(100000000039LL, sieve.primes);

```

常见坑:

- 1 不是质数。
- spf[0]、spf[1] 没有意义。
- $x > N$ 时不能用 factorize_by_spf(x)。
- 试除循环条件用 $(ll)p * p \leq x$, 避免 $p*p$ 溢出。
- factorize_by_primes 最好筛到 $\sqrt{x}$; 本模板带朴素兜底, 质数表不够时仍正确但会慢。
- 多测不同 N 时, 筛到所有测试的最大 N 更省时间。

暴力/部分替代:

- 单次判断质数: 枚举 $d=2..\sqrt{x}$ 。
- 单次分解: 从 $d=2$ 开始试除。
- $N \leq 5000$: 直接双重循环标记合数也能拿分。

升级方向:

- 区间 $[L, R]$ 很大但长度小 $\rightarrow$ 区间筛。
- 大整数质性测试 $\rightarrow$ Miller-Rabin, 低优先级。
- 分解 $1e18$ 大数 $\rightarrow$ Pollard Rho, 高难低优先级。

最小测试样例:

```

init(20)
primes = 2 3 5 7 11 13 17 19
is_prime(1) = false
is_prime(17) = true
factorize_by_spf(360) = 2^3 * 3^2 * 5^1

```

MATH-05 矩阵快速幂

模块编号: MATH-05

模块名称: 矩阵快速幂与线性递推

标签: [数学][矩阵][快速幂][递推]

一句话用途: 把固定线性递推的第 n 项从逐项推 $O(n)$ 升级到 $O(k^3 \log n)$ 。

题面触发词:

- 求斐波那契第 n 项, n 很大。
- 线性递推、第 n 天、第 n 次操作。
- 状态转移固定, 重复执行很多次。
- $n \leq 1e18$, 状态维度很小。
- 每一步从前几项线性组合得到下一项。

什么时候用:

- 状态转移可以写成 $\text{next} = T * \text{current}$ 。
- 转移矩阵不随时间变化。
- n 很大但状态维度 k 小, 一般 $k \leq 50$ 。
- 需要在模数下输出结果。

不要什么时候用:

- 转移每一步都变, 不是固定矩阵。
- 状态维度太大, k^3 扛不住。
- n 很小, 直接循环 DP 更简单。
- 转移包含 $\max/\min$, 不是普通加乘线性递推。

复杂度:

- 矩阵乘法: $O(k^3)$ 。
- 矩阵快速幂: $O(k^3 \log n)$ 。
- 矩阵乘向量: $O(k^2)$ 。

数据范围参考:

- $k \leq 30$ 且 $n \leq 1e18$: 很常用。
- $k \leq 100$: 可能还能过, 注意常数。
- $k > 300$: 普通矩阵快速幂通常不合适。

依赖的标准容器:

- `vector<vector<ll>>`。
- 状态向量按 0-index 存放; 数学模块内部下标自然从 $0..k-1$ 。

输入如何整理:

```
ll n, mod;
cin >> n >> mod;
// 按题意构造转移矩阵 T 和初始向量 base
```

接口:

```
Matrix(k, mod)
identity(k, mod)
multiply(A,B)
mpow(A,e)
apply(A, vec)
```

输出能力:

- 线性递推第 n 项。
- 固定转移重复执行 n 次后的状态。
- 小维度路径计数: 邻接矩阵 A^k 。

下游可接:

- 快速幂。
- 图上固定步数路径计数。
- DP 的矩阵优化。

可拼接模块:

- MATH-02 快速幂与模运算。
- DP 线性递推。
- GRAPH 邻接矩阵路径计数。

模数是否为质数的分支:

矩阵快速幂只做加法和乘法, mod 是否为质数都可以。

如果矩阵推导中需要除法或逆矩阵:

mod 是质数 -> 可考虑费马逆元。

mod 不是质数 -> 只有互质元素才可逆，考场低优先级，尽量改推导避免除法。

模板代码：

```
using ll = long long;
```

```
struct Matrix {
    int n;
    ll mod;
    vector<vector<ll>> a;

    Matrix(int n_ = 0, ll mod_ = 1) {
        n = n_;
        mod = mod_;
        a.assign(n, vector<ll>(n, 0));
    }
};
```

```
Matrix identity(int n, ll mod) {
    Matrix I(n, mod);
    for (int i = 0; i < n; i++) I.a[i][i] = 1 % mod;
    return I;
}
```

```
Matrix multiply(const Matrix &A, const Matrix &B) {
    int n = A.n;
    ll mod = A.mod;
    Matrix C(n, mod);
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (A.a[i][k] == 0) continue;
            for (int j = 0; j < n; j++) {
                C.a[i][j] = (C.a[i][j] + (__int128)A.a[i][k] * B.a[k][j]) % mod;
                if (C.a[i][j] < 0) C.a[i][j] += mod;
            }
        }
    }
    return C;
}
```

```
Matrix mpow(Matrix A, long long e) {
    Matrix res = identity(A.n, A.mod);
    while (e > 0) {
        if (e & 1) res = multiply(res, A);
        A = multiply(A, A);
        e >>= 1;
    }
    return res;
}
```

```
vector<ll> apply(const Matrix &A, const vector<ll> &v) {
    int n = A.n;
    vector<ll> res(n, 0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res[i] = (res[i] + (__int128)A.a[i][j] * v[j]) % A.mod;
            if (res[i] < 0) res[i] += A.mod;
        }
    }
    return res;
}
```

调用示例：

```
// Fibonacci: F0=0, F1=1
// [F_n, F_{n-1}]^T = [[1,1],[1,0]]^(n-1) * [F1,F0]^T
ll fib(ll n, ll mod) {
    if (n == 0) return 0;
    Matrix T(2, mod);
    T.a = {{1, 1}, {1, 0}};
    Matrix P = mpow(T, n - 1);
    vector<ll> base = {1, 0};
    return apply(P, base)[0];
}
```

常见坑：

- 初始向量顺序必须和转移矩阵对应。
- $n=0/1$ 等边界要先判。
- 矩阵下标模板是 0-index，不要和题目 1-index 混。
- 如果 mod 很大，乘法用 `__int128`。
- 转移不是线性的题不要硬套矩阵。

暴力/部分替代：

- $n \leq 1e7$ ：直接循环递推。
- 状态很小但不会建矩阵：先写普通 DP 拿部分分。
- 图上固定步数路径小数据：重复做 k 次 DP。

升级方向：

- 矩阵很稀疏 -> 优化乘法或用状态转移循环。
- 多个 n 查询同一矩阵 -> 预处理 $T^{(2^i)}$ 。
- 递推阶数很高 -> 线性递推优化，低优先级。

最小测试样例：

```
fib(0, 1000000007) = 0
fib(1, 1000000007) = 1
fib(10, 1000000007) = 55
```

MATH-06 容斥基础

模块编号：MATH-06

模块名称：容斥原理基础模板

标签：[数学][容斥][计数][集合]

一句话用途：把“至少满足一个条件”的计数转成若干交集计数的加减。

题面触发词：

- 至少一个、不能被任何一个、被若干数整除。
- 多个限制条件，问合法数量。
- 不含某些元素、没有某些性质。
- 求并集大小。
- 条件数量较少，通常 $m \leq 20$ 。

什么时候用：

- 每个条件单独或交集都容易计数。
- 条件数不大，可以枚举子集。
- 问“满足至少一个条件”或“避开所有条件”。
- 倍数计数： $1..n$ 中能被给定数集合中至少一个整除。

不要什么时候用：

- 条件数很大， 2^m 爆炸。
- 交集计数本身很难。
- 条件之间不是简单集合关系，容易重复或漏算。
- lcm 超过上界且未防溢出。

复杂度：

- 枚举条件子集: $O(2^m * m)$ 。
- 若每个交集 $O(1)$ 可算, 则总复杂度约 $O(2^m * m)$ 。

数据范围参考:

- $m \leq 20$: 标准子集容斥。
- $m \leq 25$: 可能勉强, 注意常数。
- $n \leq 1e18$: 倍数计数可用 n / lcm , 必须防溢出。

依赖的标准容器:

- `vector<ll> a(m)`, 容斥条件内部通常用 0-index 枚举子集。
- 若条件来自题面 1-index, 读入后可放到 `a[0..m-1]`。

输入如何整理:

```
ll n;
int m;
cin >> n >> m;
vector<ll> d(m);
for (int i = 0; i < m; i++) cin >> d[i];
```

接口:

`count_divisible_by_any(n, d) -> 1..n 中能被 d 中至少一个数整除的个数`
`count_divisible_by_none(n, d) -> 1..n 中不能被任何 d 整除的个数`

输出能力:

- 并集大小。
- 不满足任何条件的补集大小。
- 倍数类容斥计数。

下游可接:

- 组合数。
- gcd/lcm。
- DP 中排除非法条件。

可拼接模块:

- MATH-01 gcd/lcm。
- MATH-03 组合数。
- MATH-04 质因数分解。

模数是否为质数的分支:

容斥加减取模时, mod 是否为质数都可以。

只要做加法、减法、乘法, 不需要质数。

若交集计数里用组合数除法:

mod 是质数 -> `fac/ifac`。

mod 不是质数 -> Pascal 小范围或其他非质数组合方案。

模板代码:

```
using ll = long long;

ll gcd_ll(ll a, ll b) {
    while (b) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a >= 0 ? a : -a;
}

ll lcm_limit(ll a, ll b, ll limit) {
    if (a == 0 || b == 0) return 0;
    __int128 aa = a, bb = b;
    if (aa < 0) aa = -aa;
    if (bb < 0) bb = -bb;
    ll g = gcd_ll(a, b);
```

```

    aa /= g;
    __int128 lim = limit;
    ll over = (limit == LLONG_MAX ? limit : limit + 1);
    if (bb != 0 && aa > lim / bb) return over;
    __int128 res = aa * bb;
    if (res > lim) return over;
    return (ll)res;
}

ll count_divisible_by_any(ll n, const vector<ll> &d) {
    int m = (int)d.size();
    __int128 ans = 0;
    for (int mask = 1; mask < (1 << m); mask++) {
        ll l = 1;
        int bits = 0;
        bool over = false;
        for (int i = 0; i < m; i++) {
            if (mask >> i & 1) {
                if (d[i] == 0) {
                    over = true;
                    break;
                }
                bits++;
                l = lcm_limit(l, d[i], n);
                if (l > n) {
                    over = true;
                    break;
                }
            }
        }
        if (over) continue;
        if (l == 0) continue;
        if (bits & 1) ans += n / l;
        else ans -= n / l;
    }
    return (ll)ans;
}

ll count_divisible_by_none(ll n, const vector<ll> &d) {
    return n - count_divisible_by_any(n, d);
}

```

调用示例:

```

ll n = 10;
vector<ll> d = {2, 3};
cout << count_divisible_by_any(n, d) << "\n"; // 7: 2,3,4,6,8,9,10
cout << count_divisible_by_none(n, d) << "\n"; // 3: 1,5,7

```

常见坑:

- 奇数个条件加, 偶数个条件减。
- 枚举子集时 m 不能太大。
- $1 \ll m$ 当 $m \geq 31$ 会溢出 int; 本基础模板默认 $m \leq 20$ 。
- lcm 必须防溢出, 超过 n 后这个子集贡献为 0。
- $d[i] == 0$ 没有“被 0 整除”的意义, 题目若可能出现要先过滤。

暴力/部分替代:

- $n \leq 1e6$: 枚举每个数 x, 检查是否满足条件。
- m 小但不会写容斥: 用集合/布尔数组标记倍数。
- 组合容斥不会推: 先枚举所有方案并检查合法性。

升级方向:

- 条件来自质因子集合 -> 先质因数分解再枚举质因子子集。
- 容斥项中需要组合数 -> 接 MATH-03。

- 条件很多但结构特殊 -> Mobius 反演, 低优先级。

最小测试样例:

```
n=10, d=[2,3]
能被至少一个整除 = 7
不能被任何一个整除 = 3
```

```
n=12, d=[2,4]
能被至少一个整除 = 6
```

STR-01 string 常用操作

模块编号: STR-01

模块名称: C++ string 常用操作与索引转换

标签: [字符串][string][基础操作][0-index]

一句话用途: 统一字符串读入、截取、查找、拼接、排序和 0-index/1-index 转换, 减少低级错误。

索引约定:

本模块内部使用 C++ string 自然 0-index: 位置 $0..n-1$ 。

题面若给第 k 个字符, 通常是 1-index: 读入后用 $k--$ 。

子串 `substr(pos, len)` 的 `pos` 是 0-index, `len` 是长度, 不是右端点。

若题面给闭区间 $[l, r]$ 且为 1-index, 则 C++ 写 `s.substr(l-1, r-l+1)`。

题面触发词:

- 字符串处理、字符替换、统计字符。
- 子串、前缀、后缀。
- 字典序排序。
- 翻转、拼接、删除、插入。
- 大小写转换。

什么时候用:

- 题目只需要简单字符串操作, 不需要 KMP/Hash。
- 字符串长度不大, 可以直接用 `substr/find`。
- 需要把题面 1-index 位置转成 C++ 0-index。

不要什么时候用:

- n, m 很大且要重复匹配子串, `find/substr` 可能 TLE。
- 需要大量判断任意子串是否相等, 优先 Rolling Hash。
- 多模式串匹配, 优先 Trie/AC 自动机。

复杂度:

- `s.size()`: $O(1)$ 。
- `s.substr(pos, len)`: $O(len)$ 。
- `s.find(t)`: 通常可用但最坏不作为算法保证。
- 排序字符串数组: $O(\text{总比较成本} * \log n)$ 。

数据范围参考:

- $|s| \leq 1e5$: 一次线性扫描没问题。
- 重复 `substr` 复制总长度可能到 $O(n^2)$, 要小心。

依赖的标准容器:

- `string`。
- `vector<int>` / `vector<string>`。
- 字符计数常用 `array<int, 26>` 或 `vector<int>(256)`。

输入如何整理:

```
string s;
cin >> s; // 无空格字符串
```

```
string line;  
getline(cin, line); // 含空格整行, 注意先处理上一行换行
```

接口:

```
to0(pos1) -> 题面 1-index 转 0-index  
substr_lidx(s,l,r) -> 题面 1-index 闭区间子串  
count_lower(s) -> 统计小写字母  
is_prefix(s,t) -> t 是否为 s 的前缀  
is_suffix(s,t) -> t 是否为 s 的后缀
```

输出能力:

- 字符频次。
- 子串、前缀、后缀。
- 字典序比较。
- 简单模拟修改后的字符串。

下游可接:

- KMP/Z 函数。
- Trie。
- Rolling Hash。
- STR-05 Manacher。

可拼接模块:

- STR-02 KMP/Z。
- STR-03 Trie/Rolling Hash。
- STR-05 Manacher。
- DP LCS/编辑距离。

模板代码:

```
int to0(int pos1) {  
    return pos1 - 1;  
}  
  
string substr_lidx(const string &s, int l, int r) {  
    if (l < 1 || r < 1 || r > (int)s.size()) return "";  
    return s.substr(l - 1, r - l + 1);  
}  
  
array<int, 26> count_lower(const string &s) {  
    array<int, 26> cnt{};  
    for (char c : s) {  
        if ('a' <= c && c <= 'z') cnt[c - 'a']++;  
    }  
    return cnt;  
}  
  
bool is_prefix(const string &s, const string &t) {  
    if (t.size() > s.size()) return false;  
    for (int i = 0; i < (int)t.size(); i++) {  
        if (s[i] != t[i]) return false;  
    }  
    return true;  
}  
  
bool is_suffix(const string &s, const string &t) {  
    int n = (int)s.size(), m = (int)t.size();  
    if (m > n) return false;  
    for (int i = 0; i < m; i++) {  
        if (s[n - m + i] != t[i]) return false;  
    }  
    return true;  
}
```

调用示例:

```
string s = "abcdef";
int l = 2, r = 4; // 题面 1-index
cout << substr_idx(s, l, r) << "\n"; // bcd

auto cnt = count_lower(s);
cout << cnt['a' - 'a'] << "\n";
```

常见坑:

- `s[i]` 是 0-index.
- `substr(pos, len)` 第二个参数是长度, 不是右端点.
- `getline` 前如果刚用过 `cin >> x`, 要吃掉换行.
- `char` 可能是 signed, 做 ASCII 桶建议转 `unsigned char` 或用 `vector<int>(256)`.
- 循环写 `i < s.size() - 1` 时, 空串会让无符号减法出事; 先转 `int n=s.size()`.

暴力/部分替代:

- 子串匹配小数据: 从每个位置开始逐字符比较.
- 子串相等小数据: 直接 `substr` 比较.
- 多次修改小数据: 直接改 `string`.

升级方向:

- 单模式匹配大数据 -> KMP/Z.
- 多模式前缀统计 -> Trie.
- 任意子串相等 -> Rolling Hash.
- 回文子串 -> STR-05 Manacher 或中心扩展.

最小测试样例:

```
s = abcdef
题面 [2,4] -> substr_idx = bcd
is_prefix(abcdef, abc) = true
is_suffix(abcdef, def) = true
```

STR-02 KMP 与 Z 函数

模块编号: STR-02

模块名称: 单模式串匹配: KMP 与 Z 函数

标签: [字符串][KMP][Z 函数][模式匹配][0-index]

一句话用途: 在线性时间内查找模式串在文本串中的所有出现位置, 并处理前后缀匹配问题。

索引约定:

本模块内部全部使用 0-index.

返回的匹配位置是文本串 `text` 的 0-index 起点.

若题目要求输出 1-index, 输出时写 `pos + 1`.

KMP 的 `pi[i]` 表示 `s[0..i]` 的最长真前后缀长度.

`Z[i]` 表示 `s[i..]` 与 `s[0..]` 的最长公共前缀长度.

题面触发词:

- 模式串在文本串中出现几次.
- 找所有匹配位置.
- 最长相同前后缀、border.
- 字符串周期、循环节.
- 每个后缀和原串的最长公共前缀.

什么时候用:

- 单个模式串匹配一个或多个文本.
- 需要线性复杂度.
- 前后缀关系明显.
- 需要判断字符串最小周期.

不要什么时候用：

- 多个模式串同时匹配，优先 Trie/AC 自动机。
- 只是一次短字符串匹配，暴力即可。
- 任意两个子串比较，优先 Rolling Hash。

复杂度：

- KMP 前缀函数： $O(n)$ 。
- KMP 匹配： $O(n+m)$ 。
- Z 函数： $O(n)$ 。

数据范围参考：

- $|s|, |text| \leq 1e6$ ：KMP/Z 都可用。
- 多测总长度很大时，按总长度线性处理。

依赖的标准容器：

- string。
- vector<int>，下标 $0..n-1$ 。

输入如何整理：

```
string text, pat;
cin >> text >> pat;
```

接口：

```
prefix_function(s) -> pi 数组
kmp_find_all(text, pat) -> pat 在 text 中所有 0-index 起点
z_function(s) -> z 数组
minimal_period(s) -> 最小周期长度
```

输出能力：

- 模式串出现次数。
- 所有匹配位置。
- 最长 border。
- 最小周期。
- 后缀与整串的 LCP。

下游可接：

- 字符串周期 + gcd/lcm。
- DP 中的自动机状态。
- 题面要求输出 1-index 位置时接索引转换。

可拼接模块：

- STR-01 基础操作。
- STR-03 Rolling Hash。
- MATH-01 gcd/lcm。

模板代码：

```
vector<int> prefix_function(const string &s) {
    int n = (int)s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_find_all(const string &text, const string &pat) {
    vector<int> ans;
    if (pat.empty()) return ans;
    string s = pat + "#" + text;
```

```

vector<int> pi = prefix_function(s);
int m = (int)pat.size();
for (int i = m + 1; i < (int)s.size(); i++) {
    if (pi[i] == m) {
        ans.push_back(i - 2 * m); // text 中的 0-index 起点
    }
}
return ans;
}

```

```

vector<int> z_function(const string &s) {
    int n = (int)s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

```

```

int minimal_period(const string &s) {
    int n = (int)s.size();
    if (n == 0) return 0;
    vector<int> pi = prefix_function(s);
    int p = n - pi[n - 1];
    if (n % p == 0) return p;
    return n;
}

```

调用示例:

```

string text = "ababa", pat = "aba";
auto pos = kmp_find_all(text, pat);
for (int p : pos) cout << p + 1 << " "; // 题目要 1-index 时输出 1 3

```

```

string s = "ababab";
cout << minimal_period(s) << "\n"; // 2

```

常见坑:

- 返回位置是 0-index, 输出题面位置通常要 +1。
- 拼接 pat + "#" + text 时, 分隔符 # 不能出现在字符集里; 若可能出现, 换一个不存在的字符。
- pi[i] 是长度, 不是下标。
- minimal_period 要检查 n % p == 0。
- 空模式串一般题目不会给, 模板里直接返回空结果。

暴力/部分分替代:

- |text| * |pat| <= 1e7: 枚举起点逐字符比较。
- 周期小数据: 枚举周期长度 p, 检查每个字符是否等于 s[i%p]。
- 前后缀小数据: 枚举长度后比较 substr。

升级方向:

- 多模式匹配 -> AC 自动机。
- 大量子串相等/LCP 查询 -> Rolling Hash 或后缀数组, 后缀数组低优先级。
- KMP 自动机 DP -> 在 pi 基础上构建转移。

最小测试样例:

```

text=ababa, pat=aba
0-index 匹配位置: 0 2
1-index 输出位置: 1 3

```

```
s=ababab
minimal_period=2
```

STR-03 Trie 与 Rolling Hash

模块编号: STR-03

模块名称: Trie 字典树与字符串 Rolling Hash

标签: [字符串][Trie][Hash][0-index][前缀]

一句话用途: Trie 维护大量字符串前缀, Rolling Hash 用 $O(1)$ 判断子串相等。

索引约定:

Trie 逐字符扫描 string, 字符位置使用 0-index。

Rolling Hash 的字符串 s 是 0-index, 但哈希前缀数组 h/pw 使用 1-index:

h[i+1] 表示 s[0..i] 的哈希。

get(l,r) 接收 0-index 闭区间 [l,r]。

若题面给 1-index 闭区间 [l,r], 调用 get(l-1, r-1)。

题面触发词:

- 前缀、字典、单词集合。
- 插入字符串、查询某前缀出现次数。
- 大量判断两个子串是否相同。
- 最长公共前缀、字符串去重。
- 子串哈希、滚动哈希。

什么时候用:

- Trie: 很多字符串共享前缀, 或要统计前缀数量。
- Hash: 多次询问 $s[l1..r1] == s[l2..r2]$ 。
- 需要二分 LCP 时, Hash 可快速比较子串。

不要什么时候用:

- Trie 字符集很大且节点数爆炸时, 要改用 map/unordered_map 子边或排序。
- Hash 有碰撞风险, 严谨题要双哈希。
- 单次子串比较, 小数据直接 substr 更稳。
- 多模式串在文本中匹配, Trie 只能做前缀, 完整多模式匹配看 AC 自动机。

复杂度:

- Trie 插入/查询: $O(\text{字符串长度})$ 。
- Hash 预处理: $O(n)$ 。
- Hash 子串查询: $O(1)$ 。

数据范围参考:

- 总字符数 $\leq 1e6$: Trie 数组节点可用。
- $|s|, q \leq 2e5$: Rolling Hash 常用。

依赖的标准容器:

- vector<array<int,26>> 或节点结构。
- vector<ll> h, pw。
- 字符串内部 0-index, Hash 前缀数组 1-index。

输入如何整理:

```
int n;
cin >> n;
Trie trie;
trie.init();
for (int i = 1; i <= n; i++) {
    string s;
    cin >> s;
```



```
    trie.insert(s);
}
```

接口:

```
Trie.init()
Trie.insert(s)
Trie.count_word(s)
Trie.count_prefix(s)
RollingHash.build(s)
RollingHash.get(l,r) -> 0-index 闭区间哈希
RollingHash.same(l1,r1,l2,r2)
```

输出能力:

- 单词出现次数。
- 前缀出现次数。
- 子串相等判断。
- 可辅助 LCP、回文判断、去重。

下游可接:

- 字典序 DFS。
- 二分答案 + Hash。
- AC 自动机。
- 字符串 DP。

可拼接模块:

- STR-01 基础操作。
- STR-02 KMP/Z。
- STR-04 AC 自动机。
- STR-05 Manacher。
- 二分答案。

模板代码:

```
struct Trie {
    struct Node {
        array<int, 26> nxt{};
        int pass = 0;
        int end = 0;
    };

    vector<Node> tr;

    void init() {
        tr.clear();
        tr.push_back(Node());
    }

    bool valid_lowercase(const string &s) const {
        for (char c : s) {
            int x = c - 'a';
            if (x < 0 || x >= 26) return false;
        }
        return true;
    }

    void insert(const string &s) {
        if (!valid_lowercase(s)) return; // 默认小写; 其他字符集先改映射。
        int u = 0;
        tr[u].pass++;
        for (char c : s) {
            int x = c - 'a';
            if (tr[u].nxt[x] == 0) {
                tr[u].nxt[x] = (int)tr.size();
                tr.push_back(Node());
            }
        }
    }
};
```

```

    }
    u = tr[u].nxt[x];
    tr[u].pass++;
}
tr[u].end++;
}

int count_word(const string &s) const {
    int u = 0;
    for (char c : s) {
        int x = c - 'a';
        if (x < 0 || x >= 26) return 0;
        if (tr[u].nxt[x] == 0) return 0;
        u = tr[u].nxt[x];
    }
    return tr[u].end;
}

int count_prefix(const string &s) const {
    int u = 0;
    for (char c : s) {
        int x = c - 'a';
        if (x < 0 || x >= 26) return 0;
        if (tr[u].nxt[x] == 0) return 0;
        u = tr[u].nxt[x];
    }
    return tr[u].pass;
}
};

struct RollingHash {
    static const long long MOD = 1000000007LL;
    static const long long BASE = 911382323LL;
    vector<long long> h, pw;

    void build(const string &s) {
        int n = (int)s.size();
        h.assign(n + 1, 0);
        pw.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * BASE + (unsigned char)s[i] + 1) % MOD;
            pw[i + 1] = pw[i] * BASE % MOD;
        }
    }

    long long get(int l, int r) const {
        if (l > r) return 0;
        if (l < 0 || r + 1 >= (int)h.size()) return -1;
        long long res = (h[r + 1] - h[l] * pw[r - l + 1]) % MOD;
        if (res < 0) res += MOD;
        return res;
    }

    bool same(int l1, int r1, int l2, int r2) const {
        if (r1 - l1 != r2 - l2) return false;
        long long a = get(l1, r1);
        long long b = get(l2, r2);
        if (a < 0 || b < 0) return false;
        return a == b;
    }
};

```

调用示例:

```

Trie trie;
trie.init();
trie.insert("apple");
trie.insert("app");
cout << trie.count_prefix("app") << "\n"; // 2

string s = "abacaba";
RollingHash rh;
rh.build(s);
cout << rh.same(0, 2, 4, 6) << "\n"; // aba == aba

// 题面 1-index [l,r]
int l = 1, r = 3;
cout << rh.get(l - 1, r - 1) << "\n";

```

常见坑:

- Trie 模板默认只支持 'a'..'z', 其他字符要改字符映射; 模板里加了越界防御, 避免数组炸掉。
- Trie 的 0 是根节点, 所以子边用 0 表示不存在, 新节点编号从 1 开始。
- Hash 存在碰撞, 严谨时用双模数或 unsigned long long 双保险。
- Hash 的 get(l,r) 是 0-index 闭区间。
- BASE 要小于 MOD, 并尽量选大一点的随机奇数。

暴力/部分替代:

- 前缀查询小数据: 把所有单词存 vector<string>, 逐个比较前缀。
- 子串相等小数据: 直接 s.substr(1,len)==s.substr(12,len)。
- 去重小数据: 直接 set<string> 存真实子串。

升级方向:

- 多模式串在长文本中出现 -> AC 自动机。
- Hash 碰撞风险 -> 双哈希。
- 字符集大 -> Trie 子边改 map<char,int>。

最小测试样例:

```

insert apple, app
count_word(app)=1
count_prefix(app)=2

s=abacaba
same(0,2,4,6)=true
题面 [1,3] 要调用 get(0,2)

```

STR-04 AC 自动机 (低优先级)

模块编号: STR-04

模块名称: 多模式串匹配 AC 自动机

标签: [字符串][AC 自动机][多模式匹配][Trie][BFS][低优先级]

一句话用途: 很多模式串同时在一个文本里匹配时, 用 AC 自动机把 Trie 和 KMP 的失败指针思想合在一起, 避免对每个模式串分别 KMP。

索引约定:

Trie 节点编号从 0 开始, 0 是根节点。

读入的第 i 个模式串仍然按题面 1-index 编号。

扫描 text 时, text 字符位置自然是 0-index; 如果题目要输出题面位置, 通常输出 pos + 1。

字符集默认小写 a-z。不是小写字母时, 要先改映射。

回文相关不要在这里找, 直接翻 STR-05 Manacher。

题面触发词:

- 给很多模式串, 问它们在文本中出现次数。
- 敏感词过滤、关键词匹配、多关键词检索。

- 字典中任意单词是否出现在文章里。
- 多模式串总长度很大，逐个 KMP 会超时。

什么时候用：

- 模式串很多，总长度 sumLen 大，文本也长。
- 要统计所有模式串总出现次数。
- 要判断文本是否包含任意模式串。
- 要做“禁止出现某些模式串”的 DP，AC 自动机可以作为状态转移图。

不要什么时候用：

- 只有一个模式串：KMP 更短。
- 只是前缀统计：Trie 更短。
- 字符集很大且题目很简单：先考虑 map/unordered_map 或逐个 KMP 拿部分分。
- 要求每个模式串分别出现次数：基础 AC 还不够，需要额外记录终点并做 fail 树/拓扑累加。

复杂度：

- 建 Trie: $O(\text{模式串总长度})$ 。
- 建 fail: 小写 26 字符集下 $O(\text{节点数} * 26)$ 。
- 扫描文本: $O(|\text{text}|)$ ，如果要输出所有匹配位置，还要加输出量。

数据范围参考：

数据范围	建议
模式串数量少、文本短	暴力或 KMP 拿分
总模式长度 $\leq 2e5$	AC 自动机稳
总模式长度 $\leq 1e6$	静态数组 AC 更稳；本模板用 vector，写法更短

依赖的标准容器：

- string: 模式串和文本。
- queue<int>: BFS 构造 fail 指针。
- vector: 短模板写法；若总长度很大，可改成静态全局数组。

输入如何整理：

```
int m;
cin >> m;
AC ac;
ac.init();
for (int i = 1; i <= m; i++) {
    string p;
    cin >> p;
    ac.insert(p);
}
ac.build();
string text;
cin >> text;
cout << ac.count_matches(text) << "\n";
```

接口：

```
AC.init()
AC.insert(pattern)
AC.build()
AC.count_matches(text) -> 所有模式串总出现次数
AC.contains_any(text) -> 是否出现任意模式串
```

模板代码：

```
struct AC {
    struct Node {
        int nxt[26];
        int fail;
        int out;

        Node() {
```

```

        memset(nxt, 0, sizeof(nxt));
        fail = 0;
        out = 0;
    }
};

vector<Node> tr;
bool built;

void init() {
    tr.clear();
    tr.push_back(Node());
    built = false;
}

bool valid_lowercase(const string &s) const {
    for (char c : s) {
        int x = c - 'a';
        if (x < 0 || x >= 26) return false;
    }
    return true;
}

void insert(const string &s) {
    if (!valid_lowercase(s)) return;
    int u = 0;
    for (char c : s) {
        int x = c - 'a';
        if (!tr[u].nxt[x]) {
            tr[u].nxt[x] = (int)tr.size();
            tr.push_back(Node());
        }
        u = tr[u].nxt[x];
    }
    tr[u].out++;
}

void build() {
    if (built) return;
    built = true;
    queue<int> q;
    for (int c = 0; c < 26; c++) {
        int v = tr[0].nxt[c];
        if (v) q.push(v);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        tr[u].out += tr[tr[u].fail].out;
        for (int c = 0; c < 26; c++) {
            int v = tr[u].nxt[c];
            if (v) {
                tr[v].fail = tr[tr[u].fail].nxt[c];
                q.push(v);
            } else {
                tr[u].nxt[c] = tr[tr[u].fail].nxt[c];
            }
        }
    }
}

long long count_matches(const string &text) const {
    long long ans = 0;

```

```

    int u = 0;
    for (char ch : text) {
        int c = ch - 'a';
        if (c < 0 || c >= 26) {
            u = 0;
            continue;
        }
        u = tr[u].nxt[c];
        ans += tr[u].out;
    }
    return ans;
}

bool contains_any(const string &text) const {
    int u = 0;
    for (char ch : text) {
        int c = ch - 'a';
        if (c < 0 || c >= 26) {
            u = 0;
            continue;
        }
        u = tr[u].nxt[c];
        if (tr[u].out > 0) return true;
    }
    return false;
}
};

```

完整可运行代码:

```

#include <bits/stdc++.h>
using namespace std;

struct AC {
    struct Node {
        int nxt[26];
        int fail;
        int out;

        Node() {
            memset(nxt, 0, sizeof(nxt));
            fail = 0;
            out = 0;
        }
    };

    vector<Node> tr;
    bool built;

    void init() {
        tr.clear();
        tr.push_back(Node());
        built = false;
    }

    bool valid_lowercase(const string &s) const {
        for (char c : s) {
            int x = c - 'a';
            if (x < 0 || x >= 26) return false;
        }
        return true;
    }

    void insert(const string &s) {

```

```

    if (!valid_lowercase(s)) return;
    int u = 0;
    for (char c : s) {
        int x = c - 'a';
        if (!tr[u].nxt[x]) {
            tr[u].nxt[x] = (int)tr.size();
            tr.push_back(Node());
        }
        u = tr[u].nxt[x];
    }
    tr[u].out++;
}

void build() {
    if (built) return;
    built = true;
    queue<int> q;
    for (int c = 0; c < 26; c++) {
        int v = tr[0].nxt[c];
        if (v) q.push(v);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        tr[u].out += tr[tr[u].fail].out;
        for (int c = 0; c < 26; c++) {
            int v = tr[u].nxt[c];
            if (v) {
                tr[v].fail = tr[tr[u].fail].nxt[c];
                q.push(v);
            } else {
                tr[u].nxt[c] = tr[tr[u].fail].nxt[c];
            }
        }
    }
}

long long count_matches(const string &text) const {
    long long ans = 0;
    int u = 0;
    for (char ch : text) {
        int c = ch - 'a';
        if (c < 0 || c >= 26) {
            u = 0;
            continue;
        }
        u = tr[u].nxt[c];
        ans += tr[u].out;
    }
    return ans;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int m;
    cin >> m;
    AC ac;
    ac.init();
    for (int i = 1; i <= m; i++) {
        string p;

```

```

        cin >> p;
        ac.insert(p);
    }
    ac.build();

    string text;
    cin >> text;
    cout << ac.count_matches(text) << "\n";
    return 0;
}

```

调用示例:

```

AC ac;
ac.init();
ac.insert("he");
ac.insert("she");
ac.insert("hers");
ac.build();
cout << ac.count_matches("shers") << "\n"; // 3

```

常见坑:

- 本模板默认只支持小写字母; 字符集不同要改 `nxt[26]` 和 `c - 'a'`。
- `out` 通过 `fail` 累加后, `count_matches` 统计的是所有模式总出现次数。
- 如果题目问“每个模式串分别出现几次”, 要保存每个模式的终点节点, 扫描后对节点访问次数沿 `fail` 树累加。
- 如果模式串里有重复, `out++` 会把重复模式也计入总匹配次数。
- 文本中遇到非小写字符时, 本模板回到根节点。

暴力/部分替代:

- 模式串少: 每个模式暴力匹配或 KMP。
- 只问是否出现任意一个: 逐个 `text.find(pattern)` 先拿小数据。
- 字符集复杂: 先用 `unordered_map<char,int>` 写 Trie 或直接暴力拿分。

升级方向:

- AC + DP: 统计不含敏感词的字符串数量。
- AC + `fail` 树: 统计每个模式串出现次数。
- AC + 拓扑累加: 把扫描命中的节点次数传给 `fail` 祖先。

最小测试样例:

输入:

```

3
he
she
hers
shers

```

输出:

```

3

```

STR-05 Manacher 回文算法

模块编号: STR-05

模块名称: Manacher 回文半径、最长回文子串、区间回文判断

标签: [字符串][回文][Manacher][最长回文子串][1-index]

一句话用途: 在线性时间 $O(n)$ 求每个中心能扩展出的最长回文半径, 适合长字符串的最长回文子串、统计回文子串数量、大量区间回文判断。

索引约定:

本模板把原字符串 `raw` 转成 1-index 字符串 `s = " " + raw`。
`s[1..n]` 是真实字符。

$d1[i]$: 以 i 为中心的奇数回文半径。
覆盖区间: $[i - d1[i] + 1, i + d1[i] - 1]$ 。
对应最长奇数回文长度: $2 * d1[i] - 1$ 。

$d2[i]$: 以 $i-1$ 和 i 中间为中心的偶数回文半径。
覆盖区间: $[i - d2[i], i + d2[i] - 1]$ 。
对应最长偶数回文长度: $2 * d2[i]$ 。

题面如果给 1-index 区间 $[l, r]$, 直接调用 `is_pal(l, r)`。

题面触发词:

- 最长回文子串。
- 回文半径。
- 回文子串数量。
- 很多次询问 $s[l..r]$ 是否为回文。
- 字符串长度很大, 中心扩展 $O(n^2)$ 可能超时。

什么时候用:

- n 到 $1e5$ 、 $1e6$ 级别, 需要处理所有回文中心。
- 回文查询次数很多, 不能每次双指针检查。
- 需要把“某段是不是回文”作为 DP 或枚举的快速判断条件。

不要什么时候用:

- 只判断一个字符串整体是否回文: 直接双指针或 `reverse`。
- 只做几次短区间回文判断: 直接检查更快写。
- 需要动态修改字符串后再查回文: Manacher 是静态预处理, 修改后要重建。
- 题目是“最长回文子序列”: 那是 DP, 不是 Manacher。

复杂度:

- 预处理: $O(n)$ 。
- 最长回文子串: 预处理后 $O(n)$ 扫一遍。
- 单次区间回文判断: $O(1)$ 。
- 统计回文子串数量: $O(n)$, 答案可能需要 long long。

数据范围参考:

数据范围	建议
$n \leq 2000$	中心扩展或区间 DP 都能尝试
$n \leq 1e5$	Manacher 稳
$n \leq 1e6$	Manacher + 静态全局数组, 避免反复分配

依赖的标准容器:

- `string`: 存原串和 1-index 处理串。
- 静态全局数组 $d1[]/d2[]$: 存奇偶回文半径。

输入如何整理:

```
string raw;  
cin >> raw;  
build_manacher(raw);
```

接口:

`build_manacher(raw)`: 预处理 $d1/d2$ 。
`is_pal(l, r)`: 判断 1-index 闭区间 $[l, r]$ 是否为回文。
`longest_pal_len()`: 返回最长回文子串长度。
`count_pal_substrings()`: 返回回文子串总数。

模板代码:

```
const int MAXN = 1000000 + 5;  
  
int n;  
string s;           // s[1..n]  
int d1[MAXN];       // odd radius
```

```

int d2[MAXN];    // even radius, center between i-1 and i

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) {
            k++;
        }
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) {
            k++;
        }
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

bool is_pal(int l, int r) {
    if (l > r) return true;
    if (l < 1 || r > n) return false;
    int len = r - l + 1;
    if (len & 1) {
        int mid = (l + r) / 2;
        return d1[mid] >= len / 2 + 1;
    } else {
        int mid = (l + r + 1) / 2;
        return d2[mid] >= len / 2;
    }
}

int longest_pal_len() {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, 2 * d1[i] - 1);
        ans = max(ans, 2 * d2[i]);
    }
    return ans;
}

long long count_pal_substrings() {
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        ans += d1[i];
        ans += d2[i];
    }
    return ans;
}

```

```
}
```

完整可运行代码 1: 最长回文子串长度

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000 + 5;

int n;
string s;
int d1[MAXN], d2[MAXN];

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) k++;
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

int longest_pal_len() {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, 2 * d1[i] - 1);
        ans = max(ans, 2 * d2[i]);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string raw;
    cin >> raw;
    build_manacher(raw);
    cout << longest_pal_len() << "\n";
    return 0;
}
```

完整可运行代码 2: 多次判断区间是否回文

```
#include <bits/stdc++.h>
using namespace std;
```

```

const int MAXN = 1000000 + 5;

int n;
string s;
int d1[MAXN], d2[MAXN];

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) k++;
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

bool is_pal(int l, int r) {
    if (l > r) return true;
    if (l < 1 || r > n) return false;
    int len = r - l + 1;
    if (len & 1) {
        int mid = (l + r) / 2;
        return d1[mid] >= len / 2 + 1;
    }
    int mid = (l + r + 1) / 2;
    return d2[mid] >= len / 2;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string raw;
    cin >> raw;
    build_manacher(raw);

    int q;
    cin >> q;
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << (is_pal(l, r) ? "YES" : "NO") << "\n";
    }
    return 0;
}

```

```
}
```

调用示例:

```
string raw = "abacaba";
build_manacher(raw);
cout << longest_pal_len() << "\n";           // 7
cout << is_pal(1, 7) << "\n";               // true
cout << count_pal_substrings() << "\n";      // 所有回文子串个数
```

常见坑:

- $d1[i]$ 和 $d2[i]$ 是半径, 不是长度。
- 奇数回文中心是一个字符 i ; 偶数回文中心是缝隙, 在 $i-1$ 和 i 之间。
- 题面如果给的是 1-index 区间, 本模板可以直接用; 不要再 $l--$, $r--$ 。
- 空格、中文等复杂字符按字节处理; 算法竞赛通常是小写字母或 ASCII 字符。
- $\text{count_pal_substrings}()$ 可能到 $n*(n+1)/2$, 必须用 long long。
- Manacher 只能处理静态字符串, 字符串修改后必须重建。

暴力/部分替代:

```
bool slow_pal(const string &raw, int l, int r) {
    // raw 是普通 0-index string, 题面 [l,r] 是 1-index。
    l--;
    r--;
    while (l < r) {
        if (raw[l] != raw[r]) return false;
        l++;
        r--;
    }
    return true;
}
```

- $n \leq 2000$: 可以枚举中心向两边扩展, 求最长回文。
- q 很小: 每次双指针判断区间, 先拿部分分。
- 题目是 “最少切成若干回文串”: 先用 Manacher 或 slow_pal 得到 $\text{is_pal}(l,r)$, 再接 DP。

最小测试样例 1:

输入:
babad

输出:
3

最小测试样例 2:

输入:
abacaba
4
1 7
2 4
2 6
3 5

输出:
YES
NO
YES
YES

SIM-01 模拟、字符串扫描与高精度

模块编号: SIM-01

模块名称: 模拟、字符串扫描与高精度

标签：模拟、字符串、高精度、大整数、非负整数、考场模板

一句话用途：题面让你“按规则一步一步做”或整数超过 `long long` 时，用字符串和稳定循环先拿分。

题面触发词：

- 模拟过程、按顺序执行操作、每一轮变化。
- 字符串表示的数字、位数很大、结果很大。
- 高精度加法、高精度减法、高精度乘小整数。
- 不能使用内置大整数。

什么时候用：

- 规则直接，按题面顺序执行即可。
- 数字长度可能超过 18 位，`long long` 会溢出。
- 只涉及非负大整数的加、减、比较、乘小整数。
- 需要把字符逐个扫描，统计、替换、进位或借位。

不要什么时候用：

- 大整数需要乘大整数、除法、取模很多次，本模块只提供基础版本。
- 题目本质是 DP、图论、贪心，模拟只是读入或输出辅助。
- 数字长度很小且保证在 `long long` 内，直接整数更快更短。

复杂度：

- 字符串扫描： $O(n)$ 。
- 大整数比较： $O(\text{len})$ 。
- 大整数加法： $O(\text{len})$ 。
- 大整数减法： $O(\text{len})$ ，要求 $a \geq b$ 。
- 大整数乘小整数： $O(\text{len} * \text{位运算常数})$ ，通常记 $O(\text{len})$ 。

数据范围参考：

- 位数 $\leq 1e5$ ：字符串高精度可用。
- 操作次数很多时，总复杂度按“总位数扫描次数”估算。
- 乘数 k 用 `long long` 存，题面保证是小整数时使用。

依赖的标准容器：

- `string`：存非负大整数。
- `vector<int>`：存操作、方向或状态，数组默认 1-index。

输入如何整理：

```
string a, b;
cin >> a >> b;
a = strip0(a);
b = strip0(b);

int n;
cin >> n;
vector<int> op(n + 1);
for (int i = 1; i <= n; i++) cin >> op[i];
```

模拟整理顺序：

1. 把题面状态写成变量或数组。
2. 把每一步操作写成一个循环。
3. 把边界判断写成 `inside/check` 函数。
4. 字符串数字先 `strip0`，再比较或计算。

接口：

```
strip0(s) -> 去掉前导零，空结果返回 "0"。
cmp_big(a,b) -> 比较非负大整数，返回 -1/0/1。
add_big(a,b) -> 非负大整数加法。
sub_big(a,b) -> 非负大整数减法，要求  $a \geq b$ 。
mul_small(a,k) -> 非负大整数乘 long long 小整数  $k$ ， $k$  可为负。
inside(x,y,n,m) -> 网格模拟边界判断。
```

输出能力：

- 输出模拟后的状态。
- 输出大整数计算结果。
- 输出比较结果。

下游可接：

- STR-01 基础字符串操作。
- BASIC-00 控制结构。
- BRUTE 部分模拟。
- MATH 快速幂或取模模块。

可拼接模块：

- 大整数输入接 strip0。
- 高精度加减接模拟计数。
- 字符串扫描接 KMP/Hash 前的预处理。
- 小数据模拟接暴力部分。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

string strip0(string s) {
    int i = 0;
    while (i + 1 < (int)s.size() && s[i] == '0') {
        i++;
    }
    s = s.substr(i);
    if (s.empty()) return "0";
    return s;
}

int cmp_big(string a, string b) {
    a = strip0(a);
    b = strip0(b);
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -1 : 1;
    }
    if (a == b) return 0;
    return a < b ? -1 : 1;
}

string add_big(string a, string b) {
    a = strip0(a);
    b = strip0(b);
    int i = (int)a.size() - 1;
    int j = (int)b.size() - 1;
    int carry = 0;
    string res;

    while (i >= 0 || j >= 0 || carry) {
        int x = carry;
        if (i >= 0) x += a[i--] - '0';
        if (j >= 0) x += b[j--] - '0';
        res.push_back(char('0' + x % 10));
        carry = x / 10;
    }

    reverse(res.begin(), res.end());
    return strip0(res);
}

string sub_big(string a, string b) {
```

```

a = strip0(a);
b = strip0(b);
int i = (int)a.size() - 1;
int j = (int)b.size() - 1;
int borrow = 0;
string res;

while (i >= 0) {
    int x = (a[i] - '0') - borrow;
    int y = (j >= 0 ? b[j] - '0' : 0);
    if (x < y) {
        x += 10;
        borrow = 1;
    } else {
        borrow = 0;
    }
    res.push_back(char('0' + (x - y)));
    i--;
    j--;
}

reverse(res.begin(), res.end());
return strip0(res);
}

string mul_small(string a, ll k) {
    a = strip0(a);
    if (a == "0" || k == 0) return "0";

    __int128 mag = k;
    bool neg = false;
    if (mag < 0) {
        neg = true;
        mag = -mag;
    }

    __int128 carry = 0;
    string res;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        __int128 cur = (__int128)(a[i] - '0') * mag + carry;
        res.push_back(char('0' + cur % 10));
        carry = cur / 10;
    }
    while (carry > 0) {
        res.push_back(char('0' + carry % 10));
        carry /= 10;
    }

    reverse(res.begin(), res.end());
    string ans = strip0(res);
    return neg ? "-" + ans : ans;
}

bool inside(int x, int y, int n, int m) {
    return 1 <= x && x <= n && 1 <= y && y <= m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string op;
    cin >> op;

```



```

if (op == "add") {
    string a, b;
    cin >> a >> b;
    cout << add_big(a, b) << '\n';
} else if (op == "sub") {
    string a, b;
    cin >> a >> b;
    if (cmp_big(a, b) < 0) {
        cout << '-' << sub_big(b, a) << '\n';
    } else {
        cout << sub_big(a, b) << '\n';
    }
} else if (op == "mul") {
    string a;
    ll k;
    cin >> a >> k;
    cout << mul_small(a, k) << '\n';
} else if (op == "cmp") {
    string a, b;
    cin >> a >> b;
    cout << cmp_big(a, b) << '\n';
}

return 0;
}

```

调用示例:

```

string a = "000999";
string b = "1";

cout << strip0(a) << '\n';           // 999
cout << add_big(a, b) << '\n';       // 1000
cout << sub_big("1000", "1") << '\n'; // 999
cout << mul_small("123", 45) << '\n'; // 5535
cout << cmp_big("0010", "9") << '\n'; // 1

```

// 按字符模拟进位或统计。

```

string s;
cin >> s;
int cnt = 0;
for (int i = 0; i < (int)s.size(); i++) {
    if (isdigit((unsigned char)s[i])) cnt++;
}

```

常见坑:

- 高精度数字必须当字符串读，不能先读进 long long。
- 减法模板要求 $a \geq b$ ；如果题目可能为负，要先比较并输出负号。
- `strip0("0000")` 必须返回 "0"。
- 字符转数字用 `c - '0'`，数字转字符用 `char('0' + x)`。
- 乘小整数时 k 的绝对值和进位用 `__int128`，避免 `LLONG_MIN` 取负和中间乘法溢出。
- 字符串下标是 0-index，普通数组按本书约定优先 1-index。
- `isdigit` 传入非 ASCII 字符时建议转 `unsigned char`。
- 多次模拟操作时，每组数据要重置状态和答案。

暴力/部分替代:

- 位数 ≤ 18 的子任务可先用 long long。
- 乘小整数很小，可用重复加法拿小数据。
- 复杂模拟不会优化时，先按题面逐步执行，拿小范围分。
- 大整数只需要比较大小时，不要写完整加减，先写 `strip0 + cmp_big`。

升级方向:

long long 溢出 -> string 高精度

重复加法乘小数 -> mul_small

大量取模 -> 边读边取模

大整数乘大整数 -> 另写 $O(nm)$ 乘法或 FFT, 低优先级

最小测试样例:

输入

add

999

1

输出

1000

补充自测:

cmp 0010 9 -> 1

sub 1000 1 -> 999

mul 123 45 -> 5535

add 0000 000 -> 0

SIM-02 手写高精度 BigInteger 类

模块编号: SIM-02

模块名称: 带符号高精度整数 BigInteger 类

标签: 高精度、大整数、BigInteger、运算符重载、加减乘除模、C++17、考场备用

一句话用途: 当题目要求整数位数超过 long long, 且需要把高精度整数当普通整数一样做 + - * / %、比较、输入输出时, 直接抄这个类。

题面触发词:

- 高精度整数、大整数、任意长度整数。
- 不能使用 Java BigInteger / Python int。
- 结果可能有几百位、几千位。
- 需要高精度加、减、乘、除、取模、比较。
- 需要负数。

什么时候用:

- 题目需要完整整数运算, 而不是只做加法或乘小整数。
- 大整数可能为负。
- 需要多次复用同一套运算, 手写字符串函数容易乱。
- 数据范围位数中等, $O(n^2)$ 乘法和长除法可以接受。

不要什么时候用:

- 只需要非负加减或乘小整数时, SIM-01 更短、更快、更好抄。
- 位数达到 $1e5$ 且需要大量乘法/除法时, 本类太慢, 可能要 FFT/NTT 或专门算法。
- 只需要取模一个普通 long long mod, 边读边取模更短。
- 题目只比较大小, 不要抄完整类, strip0 + cmp_big 就够。

复杂度:

- 比较、取相反数、绝对值: $O(n)$ 或 $O(1)$ 。
- 加法/减法: $O(n)$ 。
- 乘法: $O(n*m)$ 。
- 除法/取模: 十进制长除法, 约 $O(n^2)$, 常数小但不适合超大位数密集除法。
- 这里 n, m 指十进制位数。

数据范围参考:

- 几百位、几千位: 本类通常可用。
- 上万位: 加减可用, 乘除要谨慎。
- 只做一次或少量乘除: 可先用本类拿分。

依赖的标准容器:

- vector<int>: 低位在前, 每个元素一位十进制数字。

- string: 输入输出。
- istream: 流输入输出。
- algorithm: 反转和比较辅助。
- cassert: 除数为 0 时防御。

输入如何整理:

```
BigInteger a, b;
cin >> a >> b;
```

支持:

普通十进制整数

+123

-123

00000123

-00000123

0

-0 // 会规范成 0

接口:

```
BigInteger x;
BigInteger x(long long);
BigInteger x(string);
```

x.to_string()

x.is_zero()

x.abs()

比较: == != < > <= >=

一元: +x, -x

四则: + - * / %

复合: += -= *= /= %=

自增自减: ++x, x++, --x, x--

输入输出: cin >> x, cout << x

输出能力:

- 输出带符号十进制整数。
- 除法向 0 取整, 和 C++ 整数除法一致。
- 取模符号跟被除数一致, 和 C++ 整数 % 一致。

下游可接:

- 模拟题、递推题、计数题的大整数输出。
- 字符串扫描。
- 暴力/部分分中需要精确大整数的场景。

可拼接模块:

- SIM-01: 轻量非负高精度。
- MATH-02: 如果只需要模普通整数, 用快速幂/取模。
- DP-20: 方案数超出 long long 且题目不取模时, dp 值可改成 BigInteger。

模型选择卡

需求	优先用
非负加减、乘小整数	SIM-01
带符号、比较、完整 + - * / %	SIM-02 BigInteger
只要 $x \bmod m$	边读边取模
大量乘大整数	低优先级, 可能要 FFT/NTT

模板代码:

```
#include <bits/stdc++.h>
using namespace std;
```

```

struct BigInteger {
    // 十进制逐位存储, d[0] 是个位。sign: -1, 0, 1.
    vector<int> d;
    int sign;

    BigInteger(long long x = 0) {
        *this = x;
    }

    BigInteger(const string &s) {
        this->read(s);
    }

    BigInteger& operator=(long long x) {
        d.clear();
        if (x == 0) {
            sign = 0;
            return *this;
        }
        sign = 1;
        unsigned long long y;
        if (x < 0) {
            sign = -1;
            y = 0ULL - (unsigned long long)x;
        } else {
            y = (unsigned long long)x;
        }
        while (y > 0) {
            d.push_back((int)(y % 10));
            y /= 10;
        }
        return *this;
    }

    void read(const string &s) {
        d.clear();
        sign = 1;

        int i = 0;
        while (i < (int)s.size() && isspace((unsigned char)s[i])) i++;
        if (i < (int)s.size() && (s[i] == '+' || s[i] == '-')) {
            if (s[i] == '-') sign = -1;
            i++;
        }
        while (i < (int)s.size() && s[i] == '0') i++;

        for (int j = (int)s.size() - 1; j >= i; j--) {
            if (!isdigit((unsigned char)s[j])) {
                throw runtime_error("bad integer literal");
            }
            d.push_back(s[j] - '0');
        }
        trim();
    }

    string to_string() const {
        if (sign == 0) return "0";
        string s;
        if (sign < 0) s.push_back('-');
        for (int i = (int)d.size() - 1; i >= 0; i--) {
            s.push_back(char('0' + d[i]));
        }
        return s;
    }
}

```

```

}

bool is_zero() const {
    return sign == 0;
}

BigInteger abs() const {
    BigInteger res = *this;
    if (res.sign < 0) res.sign = 1;
    return res;
}

void trim() {
    while (!d.empty() && d.back() == 0) d.pop_back();
    if (d.empty()) sign = 0;
}

static int abs_cmp(const BigInteger &a, const BigInteger &b) {
    if (a.d.size() != b.d.size()) {
        return a.d.size() < b.d.size() ? -1 : 1;
    }
    for (int i = (int)a.d.size() - 1; i >= 0; i--) {
        if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
    }
    return 0;
}

void abs_add(const BigInteger &b) {
    int n = max(d.size(), b.d.size());
    d.resize(n, 0);
    int carry = 0;
    for (int i = 0; i < n; i++) {
        int cur = d[i] + carry;
        if (i < (int)b.d.size()) cur += b.d[i];
        d[i] = cur % 10;
        carry = cur / 10;
    }
    if (carry) d.push_back(carry);
    if (!d.empty() && sign == 0) sign = 1;
}

// 要求 |*this| >= |b|, 只改绝对值, 不改 sign.
void abs_sub(const BigInteger &b) {
    int borrow = 0;
    for (int i = 0; i < (int)d.size(); i++) {
        int cur = d[i] - borrow - (i < (int)b.d.size() ? b.d[i] : 0);
        if (cur < 0) {
            cur += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        d[i] = cur;
    }
    trim();
}

BigInteger operator+() const {
    return *this;
}

BigInteger operator-() const {
    BigInteger res = *this;

```

```

        res.sign = -res.sign;
        return res;
    }

    BigInteger& operator+=(const BigInteger &b) {
        if (b.sign == 0) return *this;
        if (sign == 0) {
            *this = b;
            return *this;
        }
        if (sign == b.sign) {
            abs_add(b);
            return *this;
        }

        int cmp = abs_cmp(*this, b);
        if (cmp == 0) {
            d.clear();
            sign = 0;
        } else if (cmp > 0) {
            abs_sub(b);
        } else {
            BigInteger tmp = b;
            tmp.abs_sub(*this);
            *this = tmp;
        }
        return *this;
    }

    BigInteger& operator--=(const BigInteger &b) {
        *this += -b;
        return *this;
    }

    BigInteger& operator*=(const BigInteger &b) {
        if (is_zero() || b.is_zero()) {
            d.clear();
            sign = 0;
            return *this;
        }

        vector<int> res(d.size() + b.d.size() + 1, 0);
        for (int i = 0; i < (int)d.size(); i++) {
            int carry = 0;
            for (int j = 0; j < (int)b.d.size() || carry; j++) {
                int cur = res[i + j] + carry;
                if (j < (int)b.d.size()) cur += d[i] * b.d[j];
                res[i + j] = cur % 10;
                carry = cur / 10;
            }
        }

        d = res;
        sign *= b.sign;
        trim();
        return *this;
    }

    void shift10_add(int digit) {
        assert(0 <= digit && digit <= 9);
        if (sign == 0) {
            if (digit == 0) return;
            d.push_back(digit);
        }
    }

```

```

        sign = 1;
        return;
    }
    d.insert(d.begin(), digit);
    trim();
}

static pair<BigInteger, BigInteger> divmod(BigInteger a, BigInteger b) {
    if (b.is_zero()) {
        throw runtime_error("BigInteger division by zero");
    }
    if (a.is_zero()) return {BigInteger(0), BigInteger(0)};

    int qsign = a.sign * b.sign;
    int rsign = a.sign;
    a.sign = 1;
    b.sign = 1;

    if (abs_cmp(a, b) < 0) {
        a.sign = rsign;
        a.trim();
        return {BigInteger(0), a};
    }

    BigInteger q, cur;
    q.sign = 1;
    q.d.assign(a.d.size(), 0);

    for (int i = (int)a.d.size() - 1; i >= 0; i--) {
        cur.shift10_add(a.d[i]);
        int x = 0;
        while (abs_cmp(cur, b) >= 0) {
            cur.abs_sub(b);
            x++;
        }
        q.d[i] = x;
    }

    q.sign = qsign;
    q.trim();
    cur.sign = cur.d.empty() ? 0 : rsign;
    cur.trim();
    return {q, cur};
}

BigInteger& operator/=(const BigInteger &b) {
    *this = divmod(*this, b).first;
    return *this;
}

BigInteger& operator%=(const BigInteger &b) {
    *this = divmod(*this, b).second;
    return *this;
}

BigInteger& operator++() {
    *this += 1;
    return *this;
}

BigInteger operator++(int) {
    BigInteger old = *this;
    ++(*this);
}

```

```

        return old;
    }

BigInteger& operator--() {
    *this -= 1;
    return *this;
}

BigInteger operator--(int) {
    BigInteger old = *this;
    --(*this);
    return old;
}

friend bool operator<(const BigInteger &a, const BigInteger &b) {
    if (a.sign != b.sign) return a.sign < b.sign;
    if (a.sign == 0) return false;
    int cmp = abs_cmp(a, b);
    return a.sign > 0 ? cmp < 0 : cmp > 0;
}

friend bool operator==(const BigInteger &a, const BigInteger &b) {
    return a.sign == b.sign && a.d == b.d;
}

friend bool operator!=(const BigInteger &a, const BigInteger &b) {
    return !(a == b);
}

friend bool operator>(const BigInteger &a, const BigInteger &b) {
    return b < a;
}

friend bool operator<=(const BigInteger &a, const BigInteger &b) {
    return !(b < a);
}

friend bool operator>=(const BigInteger &a, const BigInteger &b) {
    return !(a < b);
}

friend BigInteger operator+(BigInteger a, const BigInteger &b) {
    a += b;
    return a;
}

friend BigInteger operator-(BigInteger a, const BigInteger &b) {
    a -= b;
    return a;
}

friend BigInteger operator*(BigInteger a, const BigInteger &b) {
    a *= b;
    return a;
}

friend BigInteger operator/(BigInteger a, const BigInteger &b) {
    a /= b;
    return a;
}

friend BigInteger operator%(BigInteger a, const BigInteger &b) {
    a %= b;

```



```
if (a > b) {
    cout << "a bigger\n";
}
```

常见坑:

- 除数不能为 0; 模板中会抛出 runtime_error, 不要依赖 assert 防御运行时输入。
- 本模板除法向 0 取整, $-1000 / 7 = -142$ 。
- 本模板取模符号跟被除数一致, $-1000 \% 7 = -6$ 。
- 十进制逐位存储很容易看懂, 但乘除不适合超大位数密集计算。
- read() 会忽略前导零, -0000 会规范成 0。
- 如果只做非负加减乘小数, 不要抄完整类, SIM-01 更短。
- 如果要和普通整数混算, BigInteger(long long) 构造函数会自动接上, 例如 $a + 1$ 、 $a / 7$ 。

暴力/部分替代:

- 小位数子任务先用 long long。
- 只需要比较大小时, 只写字符串比较。
- 只需要 % mod 时, 边读边取模。
- 乘法不会写时, 乘小整数可用 SIM-01 mul_small。
- 除法不会写时, 小数据可重复减法先拿分。

升级方向:

long long -> SIM-01 非负字符串函数 -> SIM-02 BigInteger 类
 O(nm) 乘法 -> Karatsuba / FFT / NTT
 十进制慢除法 -> 高基数长除法
 只对普通 mod 取模 -> 边读边取模

最小测试样例:

输入

```
5
+ 999 1
- 1 1000
* -123 45
/ 1000 7
% -1000 7
```

输出

```
1000
-999
-5535
142
-6
```

补充自测:

输入

```
9
/ -1000 7
% 1000 -7
< -5 3
> 00010 9
== -0000 0
!= 123 0123
inc -1
dec 0
abs -12345
```

输出

```
-142
6
1
1
1
0
0
```

SIM-03 表达式求值与 AST 语法树

模块编号: SIM-03

模块名称: 表达式求值、递归下降解析与 AST 语法树

标签: 表达式求值、AST、递归下降、语法树、模拟、字符串扫描、C++17

一句话用途: 当题目要求计算表达式、处理括号和优先级, 或者需要对表达式结构做二次处理时, 用递归下降先把表达式建成 1-index AST, 再递归求值或做树上 DP。

题面触发词:

- 给一个算术表达式, 求它的值。
- 表达式包含括号、多位数、空格、一元负号。
- 有变量, 需要先赋值再计算。
- 要输出表达式树、前缀/后缀表达式。
- 要对表达式做替换、化简、检查、统计子表达式。
- 要多次修改变量并重复求值。

什么时候用:

- 题目不是单纯 $a+b$, 而是有优先级和括号。
- 双栈求值容易被一元负号、变量或后续扩展搞乱。
- 你需要保留表达式结构, 而不只是得到一个数。
- 后续要在表达式树上做 DFS、DP、哈希、比较或替换。

不要什么时候用:

- 表达式只含 $+ - * /$ 且只求一次值, 双栈求值更短。
- 表达式长度极大且递归深度可能接近长度, 递归下降可能栈深。
- 有浮点、小数、函数调用、数组下标、逻辑短路等复杂语法, 本模板需要扩展。
- 除法/取模涉及负数时, 要确认题目规则是否和 C++ 一致。

复杂度:

- 建树: $O(\text{len})$ 。
- 单次求值: $O(\text{节点数})$ 。
- 输出前缀/后缀表达式: $O(\text{节点数})$ 。
- 如果多次变量赋值并重复求值, 每次仍是 $O(\text{节点数})$; 可按题目做子树缓存。

数据范围参考:

- 表达式长度 $\leq 2e5$: 静态节点池开 $\text{MAXNODE} = 2 * \text{len} + 5$ 更稳。
- 普通算术表达式中节点数不超过数字/变量/运算符数量总和。
- 本模板用 `long long` 求值, 结果可能超出时改接 SIM-02 `BigInteger`。

依赖的标准容器:

- `string`: 读表达式和变量名。字符串内部按 C++ 自然 0-index。
- `map<string, long long>`: 变量赋值表, 变量少时最稳; 变量很多可改 `unordered_map`。
- 静态数组 `node[MAXNODE]`: AST 节点池, 节点编号 1-index。

输入如何整理:

```
int k;
cin >> k;
for (int i = 1; i <= k; i++) {
    string name;
    long long value;
    cin >> name >> value;
    vars[name] = value;
}
```

```
string expr;
```

```
getline(cin, expr); // 吃掉上一行换行
getline(cin, expr); // 真正表达式
```

接口:

```
Parser parser(expr);
int root = parser.parse();
long long ans = eval(root, vars);
emit_prefix(root, out);
emit_postfix(root, out);
```

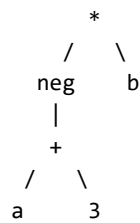
AST 节点约定:

字段	含义
type = 0	数字节点, 使用 value
type = 1	变量节点, 使用 name
type = 2	一元运算节点, 使用 op 和 left_child
type = 3	二元运算节点, 使用 op, left_child, right_child

为什么 AST 比直接求值更通用

直接求值只得到一个答案; AST 会保留结构:

表达式: $-(a + 3) * b$



保留结构之后可以继续做:

- 前缀/后缀表达式输出。
- 变量替换后重复求值。
- 统计每个子表达式的值。
- 判断两个表达式结构是否相同。
- 在表达式树上做 DP, 例如最小修改代价、布尔表达式翻转代价。

模板代码:

```
#include <cassert>
#include <cctype>
#include <climits>
#include <iostream>
#include <map>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

const int MAXNODE = 400005;

struct AstNode {
    int type; // 0 number, 1 variable, 2 unary, 3 binary
    long long value;
    string name;
    char op;
    int left_child;
    int right_child;
};

AstNode node[MAXNODE];
int node_cnt = 0;
```

```

int new_number(long long value) {
    assert(node_cnt + 1 < MAXNODE);
    ++node_cnt;
    node[node_cnt].type = 0;
    node[node_cnt].value = value;
    node[node_cnt].name.clear();
    node[node_cnt].op = 0;
    node[node_cnt].left_child = 0;
    node[node_cnt].right_child = 0;
    return node_cnt;
}

int new_variable(const string &name) {
    assert(node_cnt + 1 < MAXNODE);
    ++node_cnt;
    node[node_cnt].type = 1;
    node[node_cnt].value = 0;
    node[node_cnt].name = name;
    node[node_cnt].op = 0;
    node[node_cnt].left_child = 0;
    node[node_cnt].right_child = 0;
    return node_cnt;
}

int new_unary(char op, int child) {
    assert(node_cnt + 1 < MAXNODE);
    ++node_cnt;
    node[node_cnt].type = 2;
    node[node_cnt].value = 0;
    node[node_cnt].name.clear();
    node[node_cnt].op = op;
    node[node_cnt].left_child = child;
    node[node_cnt].right_child = 0;
    return node_cnt;
}

int new_binary(char op, int left_child, int right_child) {
    assert(node_cnt + 1 < MAXNODE);
    ++node_cnt;
    node[node_cnt].type = 3;
    node[node_cnt].value = 0;
    node[node_cnt].name.clear();
    node[node_cnt].op = op;
    node[node_cnt].left_child = left_child;
    node[node_cnt].right_child = right_child;
    return node_cnt;
}

struct Parser {
    string s;
    int pos;

    Parser(const string &expr) {
        s = expr;
        pos = 0;
    }

    void skip_spaces() {
        while (pos < (int)s.size() && isspace((unsigned char)s[pos])) pos++;
    }

    bool match(char c) {

```

```

    skip_spaces();
    if (pos < (int)s.size() && s[pos] == c) {
        pos++;
        return true;
    }
    return false;
}

int parse() {
    int root = parse_add_sub();
    skip_spaces();
    if (pos != (int)s.size()) {
        throw runtime_error("unexpected character");
    }
    return root;
}

int parse_add_sub() {
    int u = parse_mul_div_mod();
    while (true) {
        skip_spaces();
        if (pos >= (int)s.size() || (s[pos] != '+' && s[pos] != '-')) break;
        char op = s[pos++];
        int v = parse_mul_div_mod();
        u = new_binary(op, u, v);
    }
    return u;
}

int parse_mul_div_mod() {
    int u = parse_unary();
    while (true) {
        skip_spaces();
        if (pos >= (int)s.size() || (s[pos] != '*' && s[pos] != '/' && s[pos] != '%')) break;
        char op = s[pos++];
        int v = parse_unary();
        u = new_binary(op, u, v);
    }
    return u;
}

int parse_unary() {
    skip_spaces();
    if (match('+')) return parse_unary();
    if (match('-')) return new_unary('-', parse_unary());
    return parse_primary();
}

int parse_primary() {
    skip_spaces();
    if (match('(')) {
        int u = parse_add_sub();
        if (!match(' ')) {
            throw runtime_error("missing right parenthesis");
        }
        return u;
    }

    if (pos < (int)s.size() && isdigit((unsigned char)s[pos])) {
        __int128 value = 0;
        while (pos < (int)s.size() && isdigit((unsigned char)s[pos])) {
            value = value * 10 + (s[pos] - '0');
            if (value > LLONG_MAX) {

```

```

        throw runtime_error("integer literal overflow");
    }
    pos++;
}
return new_number((long long)value);
}

if (pos < (int)s.size() && (isalpha((unsigned char)s[pos]) || s[pos] == '_')) {
    string name;
    while (pos < (int)s.size()) {
        unsigned char ch = (unsigned char)s[pos];
        if (!isalnum(ch) && s[pos] != '_') break;
        name.push_back(s[pos]);
        pos++;
    }
    return new_variable(name);
}

throw runtime_error("bad primary expression");
}
};

long long checked_ll(__int128 x) {
    if (x < (__int128)LLONG_MIN || x > (__int128)LLONG_MAX) {
        throw runtime_error("integer overflow");
    }
    return (long long)x;
}

long long eval_ast(int u, const map<string, long long> &vars) {
    if (node[u].type == 0) return node[u].value;
    if (node[u].type == 1) {
        auto it = vars.find(node[u].name);
        if (it == vars.end()) {
            throw runtime_error("undefined variable");
        }
        return it->second;
    }
    if (node[u].type == 2) {
        long long x = eval_ast(node[u].left_child, vars);
        if (node[u].op == '-' && x == LLONG_MIN) {
            throw runtime_error("integer overflow");
        }
        if (node[u].op == '-') return -x;
        return x;
    }

    long long a = eval_ast(node[u].left_child, vars);
    long long b = eval_ast(node[u].right_child, vars);
    if (node[u].op == '+') return checked_ll((__int128)a + b);
    if (node[u].op == '-') return checked_ll((__int128)a - b);
    if (node[u].op == '*') return checked_ll((__int128)a * b);
    if (node[u].op == '/') {
        if (b == 0) throw runtime_error("division by zero");
        if (a == LLONG_MIN && b == -1) throw runtime_error("integer overflow");
        return a / b;
    }
    if (node[u].op == '%') {
        if (b == 0) throw runtime_error("modulo by zero");
        if (a == LLONG_MIN && b == -1) throw runtime_error("integer overflow");
        return a % b;
    }
    throw runtime_error("bad operator");
}

```

```

}

void emit_prefix(int u, vector<string> &out) {
    if (node[u].type == 0) {
        out.push_back(to_string(node[u].value));
        return;
    }
    if (node[u].type == 1) {
        out.push_back(node[u].name);
        return;
    }
    if (node[u].type == 2) {
        out.push_back("neg");
        emit_prefix(node[u].left_child, out);
        return;
    }
    out.push_back(string(1, node[u].op));
    emit_prefix(node[u].left_child, out);
    emit_prefix(node[u].right_child, out);
}

void emit_postfix(int u, vector<string> &out) {
    if (node[u].type == 0) {
        out.push_back(to_string(node[u].value));
        return;
    }
    if (node[u].type == 1) {
        out.push_back(node[u].name);
        return;
    }
    if (node[u].type == 2) {
        emit_postfix(node[u].left_child, out);
        out.push_back("neg");
        return;
    }
    emit_postfix(node[u].left_child, out);
    emit_postfix(node[u].right_child, out);
    out.push_back(string(1, node[u].op));
}

void write_tokens(const vector<string> &tokens) {
    for (int i = 0; i < (int)tokens.size(); i++) {
        if (i) cout << ' ';
        cout << tokens[i];
    }
    cout << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int k;
    if (!(cin >> k)) return 0;

    map<string, long long> vars;
    for (int i = 1; i <= k; i++) {
        string name;
        long long value;
        cin >> name >> value;
        vars[name] = value;
    }
}

```



```

string expr;
getline(cin, expr);
string line;
while (getline(cin, line)) {
    if (!expr.empty()) expr.push_back(' ');
    expr += line;
}

try {
    Parser parser(expr);
    int root = parser.parse();
    cout << eval_ast(root, vars) << '\n';
    cout << node_cnt << '\n';

    vector<string> prefix;
    emit_prefix(root, prefix);
    write_tokens(prefix);

    vector<string> postfix;
    emit_postfix(root, postfix);
    write_tokens(postfix);
} catch (const exception &e) {
    cout << "ERROR\n";
}

return 0;
}

```

调用示例:

```

map<string, long long> vars;
vars["a"] = 10;
vars["b"] = 4;

Parser parser("-(a + 3) * b");
int root = parser.parse();
cout << eval_ast(root, vars) << '\n';

```

常见坑:

- 字符串扫描天然 0-index, 这是局部例外; AST 节点编号仍是 1-index。
- 一元负号和二元减号必须分开处理: -3 是 unary, a-b 是 binary。
- parse_unary() 要放在乘除模下面、括号/数字/变量上面。
- C++ 整数除法向 0 取整, 负数 / 和 % 规则要和题面核对。
- long long 可能溢出; 表达式结果很大时把 eval_ast 的返回值改成 BigInteger。
- MAXNODE 要按表达式长度防御性开大, 普通表达式可开 $2 * \text{len} + 5$ 。
- 如果表达式有 ^, 要单独加一层优先级, 且通常是右结合。
- 如果题目有函数调用如 max(a,b), 需要在 parse_primary() 里扩展。

暴力/部分替代:

- 只含数字、+ - * /、括号且只求值: 可以先写双栈直接求值。
- 没有括号且只有 + -: 直接从左到右扫描。
- 只有单字符变量: 变量名可用数组 val[256], 不用 map。
- 需要多次换变量求值: AST 建一次, 之后每次只更新 vars 并 DFS 求值。
- 不会写完整 AST 时, 先支持无变量、无一元负号的版本拿部分分。

最小测试样例:

```

输入
2
a 10
b 4
-(a + 3) * b + 20 / 3

```

```

输出
-46

```

```

10
+ * neg + a 3 b / 20 3
a 3 + neg b * 20 3 / +

补充自测:

输入
3
x -7
y 3
z 10
x / y + x % y + z * (2 + 3)

输出
47
13
++ / x y % x y * z + 2 3
x y / x y % + z 2 3 + * +

```

SIM-04 JSON / CSV / INI 解析器

模块编号: SIM-04

模块名称: 常见数据格式解析器: JSON、CSV、INI

标签: 模拟、解析器、JSON、CSV、INI、字符串扫描、配置文件、日志处理、C++17

一句话用途: 当模拟题给出结构化文本、配置文件、表格或日志片段时, 直接套这个模块解析成树、表或键值表, 再接统计/查询逻辑。

题面触发词:

- 给一个 JSON 字符串, 查询字段、统计节点、输出某些路径。
- 给一个 CSV 表格, 处理逗号、双引号、转义引号。
- 给一个 INI 配置, 解析 section、key、value。
- 题目让你“按格式读入一段配置/日志/表格”。
- 输入不是普通空格分隔, 而是半结构化文本。

什么时候用:

- 题目重点是格式解析和模拟, 而不是算法复杂度。
- 手写 cin >> 会丢空格、逗号、引号或换行。
- JSON 需要保留结构, 后续可能查询路径或对子树统计。
- CSV 里字段可能包含逗号或引号。
- INI 里有 section 和键值对。

不要什么时候用:

- 输入格式只是普通整数/字符串列表, 直接 cin 更短。
- JSON 要求严格标准全部边角、浮点高精度或超大嵌套深度时, 本模板需要按题意微调。
- CSV 方言很复杂, 例如自定义分隔符、多行字段特殊规则, 本模板需要微调。
- INI 有复杂转义、继承、数组, 本模板只解析 [section] 下的 key=value 或 key:value。

复杂度:

- JSON 解析: $O(len)$ 。
- CSV 解析: $O(len)$ 。
- INI 解析: $O(len)$ 。
- 查询 JSON 路径若每次在线找 key, 按对象子节点数线性; 查询多时可对对象额外建 map。

数据范围参考:

- 文本长度 $\leq 2e5$: 本模块可直接用。
- JSON 节点池 MAXJSON 按文本长度防御性开大, 通常 $MAXJSON = 400005$ 。
- CSV/INI 用 vector 存结果, 行列特别大时注意内存。

依赖的标准容器:

- string: 保存原文本、字段、key、value。
- vector: 保存 CSV 表格、INI 键值、JSON 子节点。

- 静态数组 json_node[MAXJSON]: JSON AST 节点池, 节点编号 1-index.

输入如何整理:

```
string mode;
getline(cin, mode);

string text, line;
while (getline(cin, line)) {
    text += line;
    text += '\n';
}
```

接口:

JSON:

```
JsonParser parser(text);
int root = parser.parse();
dump_json_leaves(root, "$", out);
```

CSV:

```
vector<vector<string>> rows = parse_csv(text);
rows[i][j] 使用 1-index, rows[0] 和每行 [0] 是占位。
```

INI:

```
vector<IniKV> kvs = parse_ini(text);
kvs[i].section, kvs[i].key, kvs[i].value.
```

模块选择卡

格式	典型特征	输出结构
JSON	{}, [], "key", true/null	AST 树
CSV	逗号分隔, 字段可用 " 包住	1-index 表格
INI	[section], key=value	键值列表

模板代码:

```
#include <cassert>
#include <cctype>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

const int MAXJSON = 400005;

enum JsonType {
    JSON_NULL = 0,
    JSON_BOOL = 1,
    JSON_NUMBER = 2,
    JSON_STRING = 3,
    JSON_ARRAY = 4,
    JSON_OBJECT = 5
};

struct JsonNode {
    int type;
    string value;
    vector<int> child;    // 1-index: child[1..]
    vector<string> key;   // object 用, 1-index: key[1..]
};

JsonNode json_node[MAXJSON];
```

```

int json_cnt = 0;

int new_json_node(int type, const string &value = "") {
    assert(json_cnt + 1 < MAXJSON);
    ++json_cnt;
    json_node[json_cnt].type = type;
    json_node[json_cnt].value = value;
    json_node[json_cnt].child.clear();
    json_node[json_cnt].key.clear();
    json_node[json_cnt].child.push_back(0);
    json_node[json_cnt].key.push_back("");
    return json_cnt;
}

string trim_copy(const string &s) {
    int l = 0;
    int r = (int)s.size() - 1;
    while (l <= r && isspace((unsigned char)s[l])) l++;
    while (r >= l && isspace((unsigned char)s[r])) r--;
    if (l > r) return "";
    return s.substr(l, r - l + 1);
}

string escape_visible(const string &s) {
    string t;
    for (char c : s) {
        if (c == '\\') t += "\\\\";
        else if (c == '"') t += "\\\"";
        else if (c == '\n') t += "\\n";
        else if (c == '\r') t += "\\r";
        else if (c == '\t') t += "\\t";
        else t.push_back(c);
    }
    return t;
}

string escape_cell_visible(const string &s) {
    string t;
    for (char c : s) {
        if (c == '\n') t += "\\n";
        else if (c == '\r') t += "\\r";
        else if (c == '\t') t += "\\t";
        else t.push_back(c);
    }
    return t;
}

struct JsonParser {
    string s;
    int pos;

    JsonParser(const string &text) {
        s = text;
        pos = 0;
    }

    void skip_spaces() {
        while (pos < (int)s.size() && isspace((unsigned char)s[pos])) pos++;
    }

    bool starts_with(const string &pat) {
        return s.compare(pos, pat.size(), pat) == 0;
    }
}

```

```

int hex_value(char c) {
    if ('0' <= c && c <= '9') return c - '0';
    if ('a' <= c && c <= 'f') return c - 'a' + 10;
    if ('A' <= c && c <= 'F') return c - 'A' + 10;
    throw runtime_error("bad hex digit");
}

int parse_hex4() {
    int code = 0;
    for (int i = 1; i <= 4; i++) {
        if (pos >= (int)s.size() || !isxdigit((unsigned char)s[pos])) {
            throw runtime_error("bad unicode escape");
        }
        code = code * 16 + hex_value(s[pos]);
        pos++;
    }
    return code;
}

void append_utf8(string &res, int code) {
    if (code < 0 || code > 0x10FFFF) {
        throw runtime_error("bad unicode codepoint");
    }
    if (code <= 0x7F) {
        res.push_back((char)code);
    } else if (code <= 0x7FF) {
        res.push_back((char)(0xC0 | (code >> 6)));
        res.push_back((char)(0x80 | (code & 0x3F)));
    } else if (code <= 0xFFFF) {
        res.push_back((char)(0xE0 | (code >> 12)));
        res.push_back((char)(0x80 | ((code >> 6) & 0x3F)));
        res.push_back((char)(0x80 | (code & 0x3F)));
    } else {
        res.push_back((char)(0xF0 | (code >> 18)));
        res.push_back((char)(0x80 | ((code >> 12) & 0x3F)));
        res.push_back((char)(0x80 | ((code >> 6) & 0x3F)));
        res.push_back((char)(0x80 | (code & 0x3F)));
    }
}

int parse() {
    int root = parse_value();
    skip_spaces();
    if (pos != (int)s.size()) {
        throw runtime_error("extra json characters");
    }
    return root;
}

int parse_value() {
    skip_spaces();
    if (pos >= (int)s.size()) throw runtime_error("empty json value");
    char c = s[pos];
    if (c == '{') return parse_object();
    if (c == '[') return parse_array();
    if (c == '"') return new_json_node(JSON_STRING, parse_string());
    if (c == '-' || isdigit((unsigned char)c)) return parse_number();
    if (starts_with("true")) {
        pos += 4;
        return new_json_node(JSON_BOOL, "true");
    }
    if (starts_with("false")) {

```

```

        pos += 5;
        return new_json_node(JSON_BOOL, "false");
    }
    if (starts_with("null")) {
        pos += 4;
        return new_json_node(JSON_NULL, "null");
    }
    throw runtime_error("bad json value");
}

string parse_string() {
    if (pos >= (int)s.size() || s[pos] != '"') {
        throw runtime_error("missing string quote");
    }
    pos++;
    string res;
    while (pos < (int)s.size()) {
        char c = s[pos++];
        if (c == '"') return res;
        if (c != '\\') {
            if ((unsigned char)c < 0x20) {
                throw runtime_error("unescaped control character in string");
            }
            res.push_back(c);
            continue;
        }
        if (pos >= (int)s.size()) throw runtime_error("bad escape");
        char e = s[pos++];
        if (e == '"' || e == '\\') res.push_back(e);
        else if (e == 'b') res.push_back('\b');
        else if (e == 'f') res.push_back('\f');
        else if (e == 'n') res.push_back('\n');
        else if (e == 'r') res.push_back('\r');
        else if (e == 't') res.push_back('\t');
        else if (e == 'u') {
            int code = parse_hex4();
            if (0xD800 <= code && code <= 0xDBFF) {
                if (pos + 1 >= (int)s.size() || s[pos] != '\\' || s[pos + 1] != 'u') {
                    throw runtime_error("missing low surrogate");
                }
                pos += 2;
                int low = parse_hex4();
                if (low < 0xDC00 || low > 0xDFFF) {
                    throw runtime_error("bad low surrogate");
                }
                code = 0x10000 + (code - 0xD800) * 0x400 + (low - 0xDC00);
            } else if (0xDC00 <= code && code <= 0xDFFF) {
                throw runtime_error("lone low surrogate");
            }
            append_utf8(res, code);
        } else {
            throw runtime_error("unknown escape");
        }
    }
    throw runtime_error("unterminated string");
}

int parse_number() {
    int start = pos;
    if (s[pos] == '-') pos++;
    if (pos >= (int)s.size() || !isdigit((unsigned char)s[pos])) {
        throw runtime_error("bad number");
    }
}

```

```

    if (s[pos] == '0') {
        pos++;
    } else {
        while (pos < (int)s.size() && isdigit((unsigned char)s[pos])) pos++;
    }
    if (pos < (int)s.size() && s[pos] == '.') {
        pos++;
        int digit_start = pos;
        while (pos < (int)s.size() && isdigit((unsigned char)s[pos])) pos++;
        if (pos == digit_start) throw runtime_error("bad decimal number");
    }
    if (pos < (int)s.size() && (s[pos] == 'e' || s[pos] == 'E')) {
        pos++;
        if (pos < (int)s.size() && (s[pos] == '+' || s[pos] == '-')) pos++;
        int digit_start = pos;
        while (pos < (int)s.size() && isdigit((unsigned char)s[pos])) pos++;
        if (pos == digit_start) throw runtime_error("bad exponent number");
    }
    return new_json_node(JSON_NUMBER, s.substr(start, pos - start));
}

int parse_array() {
    pos++;
    int u = new_json_node(JSON_ARRAY);
    skip_spaces();
    if (pos < (int)s.size() && s[pos] == ']') {
        pos++;
        return u;
    }
    while (true) {
        int v = parse_value();
        json_node[u].child.push_back(v);
        skip_spaces();
        if (pos < (int)s.size() && s[pos] == ',') {
            pos++;
            continue;
        }
        if (pos < (int)s.size() && s[pos] == ']') {
            pos++;
            break;
        }
        throw runtime_error("bad array");
    }
    return u;
}

int parse_object() {
    pos++;
    int u = new_json_node(JSON_OBJECT);
    skip_spaces();
    if (pos < (int)s.size() && s[pos] == '}') {
        pos++;
        return u;
    }
    while (true) {
        skip_spaces();
        string k = parse_string();
        skip_spaces();
        if (pos >= (int)s.size() || s[pos] != ':') {
            throw runtime_error("missing colon");
        }
        pos++;
        int v = parse_value();
    }
}

```

```

        json_node[u].key.push_back(k);
        json_node[u].child.push_back(v);
        skip_spaces();
        if (pos < (int)s.size() && s[pos] == ',') {
            pos++;
            continue;
        }
        if (pos < (int)s.size() && s[pos] == '}') {
            pos++;
            break;
        }
        throw runtime_error("bad object");
    }
    return u;
}
};

string json_value_repr(int u) {
    int type = json_node[u].type;
    if (type == JSON_NULL) return "null";
    if (type == JSON_BOOL) return json_node[u].value;
    if (type == JSON_NUMBER) return json_node[u].value;
    if (type == JSON_STRING) return "\"" + escape_visible(json_node[u].value) + "\"";
    if (type == JSON_ARRAY) return "[array]";
    return "{object}";
}

void dump_json_leaves(int u, const string &path, vector<pair<string, string>> &out) {
    int type = json_node[u].type;
    if (type != JSON_ARRAY && type != JSON_OBJECT) {
        out.push_back({path, json_value_repr(u)});
        return;
    }
    if (type == JSON_ARRAY) {
        for (int i = 1; i < (int)json_node[u].child.size(); i++) {
            dump_json_leaves(json_node[u].child[i], path + "[" + to_string(i) + "]", out);
        }
        return;
    }
    for (int i = 1; i < (int)json_node[u].child.size(); i++) {
        dump_json_leaves(json_node[u].child[i], path + "." + json_node[u].key[i], out);
    }
}

vector<vector<string>> parse_csv(const string &text) {
    vector<vector<string>> rows(1);
    vector<string> row(1);
    string cell;
    bool in_quote = false;
    bool just_closed_quote = false;
    bool row_has_anything = false;

    for (int i = 0; i < (int)text.size(); i++) {
        char c = text[i];
        if (c == '\\r') continue;
        row_has_anything = true;

        if (in_quote) {
            if (c == '"') {
                if (i + 1 < (int)text.size() && text[i + 1] == '"') {
                    cell.push_back('"');
                    i++;
                } else {

```



```

        in_quote = false;
        just_closed_quote = true;
    }
} else {
    cell.push_back(c);
}
continue;
}

if (c == '"' && cell.empty() && !just_closed_quote) {
    in_quote = true;
} else if (c == ',') {
    row.push_back(cell);
    cell.clear();
    just_closed_quote = false;
} else if (c == '\n') {
    row.push_back(cell);
    rows.push_back(row);
    row.assign(1, "");
    cell.clear();
    just_closed_quote = false;
    row_has_anything = false;
} else if (just_closed_quote && (c == ' ' || c == '\t')) {
    continue;
} else if (just_closed_quote) {
    throw runtime_error("bad csv after quote");
} else {
    cell.push_back(c);
    just_closed_quote = false;
}
}

if (in_quote) throw runtime_error("unterminated csv quote");
if (row_has_anything || !cell.empty() || row.size() > 1) {
    row.push_back(cell);
    rows.push_back(row);
}
return rows;
}

struct InikV {
    string section;
    string key;
    string value;
};

vector<InikV> parse_ini(const string &text) {
    vector<InikV> res(1);
    string section = "global";
    string line;

    for (int i = 0; i <= (int)text.size(); i++) {
        if (i < (int)text.size() && text[i] != '\n') {
            if (text[i] != '\r') line.push_back(text[i]);
            continue;
        }

        string cur = trim_copy(line);
        line.clear();
        if (cur.empty()) continue;
        if (cur[0] == '#' || cur[0] == ';') continue;

        if (cur.front() == '[' && cur.back() == ']') {

```

```

        section = trim_copy(cur.substr(1, (int)cur.size() - 2));
        if (section.empty()) section = "global";
        continue;
    }

    int pos = -1;
    for (int j = 0; j < (int)cur.size(); j++) {
        if (cur[j] == '=' || cur[j] == ':') {
            pos = j;
            break;
        }
    }
    if (pos == -1) continue;

    InikV item;
    item.section = section;
    item.key = trim_copy(cur.substr(0, pos));
    item.value = trim_copy(cur.substr(pos + 1));
    res.push_back(item);
}

return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string mode;
    if (!getline(cin, mode)) return 0;
    mode = trim_copy(mode);

    string text, line;
    while (getline(cin, line)) {
        text += line;
        text.push_back('\n');
    }

    try {
        if (mode == "json") {
            json_cnt = 0;
            JsonParser parser(text);
            int root = parser.parse();
            vector<pair<string, string>> out;
            dump_json_leaves(root, "$", out);
            cout << json_cnt << ' ' << out.size() << '\n';
            for (auto &item : out) {
                cout << item.first << "=" << item.second << '\n';
            }
        } else if (mode == "csv") {
            vector<vector<string>> rows = parse_csv(text);
            cout << (int)rows.size() - 1 << '\n';
            for (int i = 1; i < (int)rows.size(); i++) {
                cout << "row " << i << " cols " << (int)rows[i].size() - 1 << '\n';
                for (int j = 1; j < (int)rows[i].size(); j++) {
                    cout << "[" << i << "," << j << "]" << escape_cell_visible(rows[i][j]) << '\n';
                }
            }
        } else if (mode == "ini") {
            vector<InikV> kvs = parse_ini(text);
            cout << (int)kvs.size() - 1 << '\n';
            for (int i = 1; i < (int)kvs.size(); i++) {
                cout << kvs[i].section << "." << kvs[i].key << "=" << kvs[i].value << '\n';
            }
        }
    }
}

```

```

    }
} else {
    cout << "ERROR\n";
}
} catch (const exception &e) {
    cout << "ERROR\n";
}

return 0;
}

```

调用示例:

```

JsonParser parser(text);
int root = parser.parse();
vector<pair<string, string>> leaves;
dump_json_leaves(root, "$", leaves);

```

```

vector<vector<string>> table = parse_csv(csv_text);
vector<IniKV> kvs = parse_ini(ini_text);

```

常见坑:

- JSON 字符串必须用 `getline` 或整段读入, 不能直接 `cin >> text`.
- JSON 数字这里按字符串保存, 避免溢出; 需要算数时再转 `long long` 或接 `SIM-02`.
- JSON 的 `\uXXXX` 会转成 UTF-8; 代理对例如 `\uD83D\uDE00` 也会合并后输出。
- CSV 引号内的逗号不是分隔符, `"a,b"` 是一个字段。
- CSV 中 `"` 表示一个真实的双引号。
- INI 行首 `#` 和 `;` 当注释, 行内注释本模板不自动裁掉。
- 普通数组和 CSV 行列都用 1-index; 字符串扫描下标仍是局部 0-index。

暴力/部分替代:

- JSON 不会写完整树时, 先用字符串扫描查固定字段, 拿小数据分。
- CSV 如果题目保证没有引号, 先用 `stringstream + getline(ss, cell, ',')`。
- INI 如果没有 section, 直接按 `key=value` 切分。
- 若格式只出现一两种固定模式, 手写特判比完整解析器更快。

最小测试样例:

输入

```

json
{"b":[true,null,"x"],"a":{"n":-12,"s":"hi\n"}}

```

输出

```

8 5
$.b[1]=true
$.b[2]=null
$.b[3]="x"
$.a.n=-12
$.a.s="hi\n"

```

补充自测 1:

输入

```

csv
name,score,note
Alice,10,"hello,world"
Bob,20,"a ""quote"""

```

输出

```

3
row 1 cols 3
[1,1]=name
[1,2]=score
[1,3]=note
row 2 cols 3
[2,1]=Alice

```

```
[2,2]=10
[2,3]=hello,world
row 3 cols 3
[3,1]=Bob
[3,2]=20
[3,3]=a "quote"
```

补充自测 2:

```
输入
ini
# comment
name = root
[db]
host=localhost
port : 3306
[feature]
enabled=true

输出
4
global.name=root
db.host=localhost
db.port=3306
feature.enabled=true
```

SIM-05 手写编译器 / 解释器骨架

模块编号: SIM-05

模块名称: 小语言解释器: Tokenizer、表达式 AST、语句 AST 与执行环境

标签: 模拟、解释器、编译器、AST、Tokenizer、Parser、变量环境、if、while、print、C++17

一句话用途: 当题目设计了一个“小语言”或“脚本规则”, 要求你解释执行赋值、输出、条件、循环时, 用 tokenizer + AST + 环境表拆成三层写, 避免把所有逻辑塞进字符串扫描。

题面触发词:

- 给一段程序/脚本, 要求输出执行结果。
- 有变量、赋值、表达式、print。
- 有 if/else、while、块 {}。
- 要模拟一个简单编译器、解释器、虚拟机、规则引擎。
- 需要解析语句, 而不只是单个表达式。

什么时候用:

- 输入已经像一门小语言, 而不是普通数据。
- 语句之间有状态, 例如变量会被赋值和更新。
- 需要区分词法、语法、执行, 题面规则较多。
- SIM-03 只能处理表达式, 不够处理语句流。

不要什么时候用:

- 题面只是普通表达式求值, 优先 SIM-03。
- 题目给的是汇编式指令, 例如 ADD x y, 直接逐行模拟更短。
- 语言包含函数、递归、数组、字符串变量、作用域闭包等, 本模板需要扩展。
- 循环可能无限执行, 必须按题目要求设置步数上限或检测循环。

复杂度:

- Tokenizer: $O(\text{len})$ 。
- Parser 建 AST: $O(\text{token 数})$ 。
- 执行: 约等于实际执行语句次数; while 可能远大于源码长度。
- 表达式求值: 每次按表达式子树大小。

数据范围参考:

- 源码长度 $\leq 2e5$: 节点池按 $2 * \text{token 数}$ 防御性开。
- 变量数量不大时用 `map<string, long long>` 最稳; 变量很多可改 `unordered_map`。
- 本模板设置 `EXEC_LIMIT = 1000000` 防止死循环; 正式题按题面调整。

依赖的标准容器:

- `string`: 源码和 `token` 文本。字符串扫描局部使用 0-index。
- `vector<Token>`: `token` 序列, 手动放哨兵后按 1-index 访问。
- 静态数组: 表达式节点、语句节点均为 1-index。
- `map<string, long long>`: 变量环境。

输入如何整理:

```
string code((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
```

接口:

```
vector<Token> tokens = tokenize(code);
Parser parser(tokens);
int root = parser.parse_program();
exec_stmt(root);
```

三层结构

层	做什么	不做什么
Tokenizer	把字符流切成数字、标识符、运算符、括号、分号	不判断语句是否合法
Parser	按语法建表达式 AST 和语句 AST	不执行变量赋值
Executor	根据 AST 修改变量环境并输出	不再扫描原始字符串

本模板支持的语法

```
program      := statement*
statement    := name = expr ;
              | print expr ;
              | if (expr) statement else statement
              | while (expr) statement
              | { statement* }
              | ;

expr         := comparison
comparison   := add ( (==|!=|<|<=|>|>=) add )*
add          := mul ( (+|-) mul )*
mul          := unary ( (*|/|% ) unary )*
unary        := (+|-|!) unary | primary
primary      := number | name | (expr)
```

模板代码:

```
#include <cassert>
#include <cctype>
#include <climits>
#include <iostream>
#include <iterator>
#include <map>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

using ll = long long;

const int MAXEXPR = 400005;
const int MAXSTMT = 200005;
const ll EXEC_LIMIT = 1000000;
```

```

struct Token {
    int type; // 0 end, 1 number, 2 identifier, 3 symbol/operator
    string text;
    ll value;
};

vector<Token> tokenize(const string &s) {
    vector<Token> tok(1);
    int i = 0;
    while (i < (int)s.size()) {
        unsigned char ch = (unsigned char)s[i];
        if (isspace(ch)) {
            i++;
            continue;
        }

        if (isdigit(ch)) {
            __int128 value = 0;
            int start = i;
            while (i < (int)s.size() && isdigit((unsigned char)s[i])) {
                value = value * 10 + (s[i] - '0');
                if (value > LLONG_MAX) {
                    throw runtime_error("integer literal overflow");
                }
                i++;
            }
            tok.push_back({1, s.substr(start, i - start), (ll)value});
            continue;
        }

        if (isalpha(ch) || s[i] == '_') {
            int start = i;
            while (i < (int)s.size()) {
                unsigned char c = (unsigned char)s[i];
                if (!isalnum(c) && s[i] != '_') break;
                i++;
            }
            tok.push_back({2, s.substr(start, i - start), 0});
            continue;
        }

        if (i + 1 < (int)s.size()) {
            string two = s.substr(i, 2);
            if (two == "==" || two == "!=" || two == "<=" || two == ">=") {
                tok.push_back({3, two, 0});
                i += 2;
                continue;
            }
        }

        string one_chars = "+-*/%(){};=<>!";
        if (one_chars.find(s[i]) != string::npos) {
            tok.push_back({3, string(1, s[i]), 0});
            i++;
            continue;
        }

        throw runtime_error("bad character");
    }
    tok.push_back({0, "END", 0});
    return tok;
}

```

```

struct ExprNode {
    int type; // 0 number, 1 variable, 2 unary, 3 binary
    ll value;
    string name;
    string op;
    int left_child;
    int right_child;
};

struct StmtNode {
    int type; // 0 block, 1 assign, 2 print, 3 if, 4 while
    string name;
    int expr;
    int first_child;
    int second_child;
    vector<int> body;
};

ExprNode expr_node[MAXEXPR];
StmtNode stmt_node[MAXSTMT];
int expr_cnt = 0;
int stmt_cnt = 0;

int new_expr_number(ll value) {
    assert(expr_cnt + 1 < MAXEXPR);
    ++expr_cnt;
    expr_node[expr_cnt] = {0, value, "", "", 0, 0};
    return expr_cnt;
}

int new_expr_variable(const string &name) {
    assert(expr_cnt + 1 < MAXEXPR);
    ++expr_cnt;
    expr_node[expr_cnt] = {1, 0, name, "", 0, 0};
    return expr_cnt;
}

int new_expr_unary(const string &op, int child) {
    assert(expr_cnt + 1 < MAXEXPR);
    ++expr_cnt;
    expr_node[expr_cnt] = {2, 0, "", op, child, 0};
    return expr_cnt;
}

int new_expr_binary(const string &op, int left_child, int right_child) {
    assert(expr_cnt + 1 < MAXEXPR);
    ++expr_cnt;
    expr_node[expr_cnt] = {3, 0, "", op, left_child, right_child};
    return expr_cnt;
}

int new_stmt(int type) {
    assert(stmt_cnt + 1 < MAXSTMT);
    ++stmt_cnt;
    stmt_node[stmt_cnt].type = type;
    stmt_node[stmt_cnt].name.clear();
    stmt_node[stmt_cnt].expr = 0;
    stmt_node[stmt_cnt].first_child = 0;
    stmt_node[stmt_cnt].second_child = 0;
    stmt_node[stmt_cnt].body.clear();
    return stmt_cnt;
}

```

```

bool is_cmp_op(const string &op) {
    return op == "==" || op == "!=" || op == "<" || op == "<=" || op == ">" || op == ">=";
}

struct Parser {
    vector<Token> tok;
    int pos;

    Parser(const vector<Token> &tokens) {
        tok = tokens;
        pos = 1;
    }

    Token cur() const {
        return tok[pos];
    }

    bool match(const string &text) {
        if (cur().text == text) {
            pos++;
            return true;
        }
        return false;
    }

    void expect(const string &text) {
        if (!match(text)) throw runtime_error("unexpected token");
    }

    int parse_program() {
        int u = new_stmt(0);
        while (cur().type != 0) {
            stmt_node[u].body.push_back(parse_statement());
        }
        return u;
    }

    int parse_statement() {
        if (match(";")) {
            return new_stmt(0);
        }

        if (match("{")) {
            int u = new_stmt(0);
            while (!match("}")) {
                if (cur().type == 0) throw runtime_error("missing block end");
                stmt_node[u].body.push_back(parse_statement());
            }
            return u;
        }

        if (cur().text == "print") {
            pos++;
            int u = new_stmt(2);
            stmt_node[u].expr = parse_expr();
            expect(";");
            return u;
        }

        if (cur().text == "if") {
            pos++;
            expect("(");
            int cond = parse_expr();

```



```

        expect("(");
        int then_stmt = parse_statement();
        int else_stmt = 0;
        if (cur().text == "else") {
            pos++;
            else_stmt = parse_statement();
        }
        int u = new_stmt(3);
        stmt_node[u].expr = cond;
        stmt_node[u].first_child = then_stmt;
        stmt_node[u].second_child = else_stmt;
        return u;
    }

    if (cur().text == "while") {
        pos++;
        expect("(");
        int cond = parse_expr();
        expect("(");
        int body_stmt = parse_statement();
        int u = new_stmt(4);
        stmt_node[u].expr = cond;
        stmt_node[u].first_child = body_stmt;
        return u;
    }

    if (cur().type == 2) {
        string name = cur().text;
        pos++;
        expect("=");
        int e = parse_expr();
        expect(";");
        int u = new_stmt(1);
        stmt_node[u].name = name;
        stmt_node[u].expr = e;
        return u;
    }

    throw runtime_error("bad statement");
}

int parse_expr() {
    return parse_compare();
}

int parse_compare() {
    int u = parse_add_sub();
    while (is_cmp_op(cur().text)) {
        string op = cur().text;
        pos++;
        int v = parse_add_sub();
        u = new_expr_binary(op, u, v);
    }
    return u;
}

int parse_add_sub() {
    int u = parse_mul_div_mod();
    while (cur().text == "+" || cur().text == "-") {
        string op = cur().text;
        pos++;
        int v = parse_mul_div_mod();
        u = new_expr_binary(op, u, v);
    }
}

```

```

    }
    return u;
}

int parse_mul_div_mod() {
    int u = parse_unary();
    while (cur().text == "*" || cur().text == "/" || cur().text == "%") {
        string op = cur().text;
        pos++;
        int v = parse_unary();
        u = new_expr_binary(op, u, v);
    }
    return u;
}

int parse_unary() {
    if (cur().text == "+" || cur().text == "-" || cur().text == "!") {
        string op = cur().text;
        pos++;
        return new_expr_unary(op, parse_unary());
    }
    return parse_primary();
}

int parse_primary() {
    if (cur().type == 1) {
        ll value = cur().value;
        pos++;
        return new_expr_number(value);
    }
    if (cur().type == 2) {
        string name = cur().text;
        pos++;
        return new_expr_variable(name);
    }
    if (match("(")) {
        int u = parse_expr();
        expect(")");
        return u;
    }
    throw runtime_error("bad expression");
}
};

map<string, ll> env;
vector<ll> output_values;
ll exec_steps = 0;

void tick() {
    ++exec_steps;
    if (exec_steps > EXEC_LIMIT) throw runtime_error("execution limit exceeded");
}

ll get_var(const string &name) {
    auto it = env.find(name);
    if (it == env.end()) return 0;
    return it->second;
}

ll checked_ll(__int128 x) {
    if (x < (__int128)LLONG_MIN || x > (__int128)LLONG_MAX) {
        throw runtime_error("integer overflow");
    }
}

```

```

    return (ll)x;
}

ll eval_expr(int u) {
    ExprNode &e = expr_node[u];
    if (e.type == 0) return e.value;
    if (e.type == 1) return get_var(e.name);
    if (e.type == 2) {
        ll x = eval_expr(e.left_child);
        if (e.op == "+") return x;
        if (e.op == "-") {
            if (x == LLONG_MIN) throw runtime_error("integer overflow");
            return -x;
        }
        if (e.op == "!") return x == 0;
    }

    ll a = eval_expr(e.left_child);
    ll b = eval_expr(e.right_child);
    if (e.op == "+") return checked_ll((__int128)a + b);
    if (e.op == "-") return checked_ll((__int128)a - b);
    if (e.op == "*") return checked_ll((__int128)a * b);
    if (e.op == "/") {
        if (b == 0) throw runtime_error("division by zero");
        if (a == LLONG_MIN && b == -1) throw runtime_error("integer overflow");
        return a / b;
    }
    if (e.op == "%") {
        if (b == 0) throw runtime_error("modulo by zero");
        if (a == LLONG_MIN && b == -1) throw runtime_error("integer overflow");
        return a % b;
    }
    if (e.op == "==") return a == b;
    if (e.op == "!=") return a != b;
    if (e.op == "<") return a < b;
    if (e.op == "<=") return a <= b;
    if (e.op == ">") return a > b;
    if (e.op == ">=") return a >= b;
    throw runtime_error("bad operator");
}

void exec_stmt(int u) {
    if (u == 0) return;
    tick();
    StmtNode &st = stmt_node[u];

    if (st.type == 0) {
        for (int v : st.body) exec_stmt(v);
    } else if (st.type == 1) {
        env[st.name] = eval_expr(st.expr);
    } else if (st.type == 2) {
        output_values.push_back(eval_expr(st.expr));
    } else if (st.type == 3) {
        if (eval_expr(st.expr) != 0) exec_stmt(st.first_child);
        else exec_stmt(st.second_child);
    } else if (st.type == 4) {
        while (eval_expr(st.expr) != 0) {
            tick();
            exec_stmt(st.first_child);
        }
    }
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string code((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
    if (code.empty()) return 0;

    try {
        vector<Token> tokens = tokenize(code);
        Parser parser(tokens);
        int root = parser.parse_program();
        exec_stmt(root);
        for (ll x : output_values) {
            cout << x << '\n';
        }
    } catch (const exception &e) {
        cout << "ERROR\n";
    }

    return 0;
}

```

调用示例:

```

x = 0;
sum = 0;
while (x < 5) {
    x = x + 1;
    sum = sum + x;
}
print sum;

```

常见坑:

- 解释器题一定要先分层: 字符 -> token -> AST -> 执行, 不要边扫描边执行复杂语句。
- if/while 的条件表达式统一用非 0 为真, 0 为假。
- 未定义变量本模板默认值为 0; 如果题目要求报错, 就在 get_var() 改成 throw。
- while 可能死循环, 要按题目要求设置 EXEC_LIMIT 或检测状态。
- 这里没有局部作用域, 所有变量都是全局变量; 函数和局部变量要额外加环境栈。
- C++ 除法和取模按向 0 取整, 负数规则要和题目核对。
- 关键字 print/if/else/while 不应再当普通变量名。
- 语句 AST 节点和表达式 AST 节点都是 1-index; token vector 也从 1 开始放有效 token。

暴力/部分替代:

- 如果只有逐行指令, 例如 SET x 1、ADD x 2, 直接按行模拟, 不必建 AST。
- 如果没有 if/while, 只写赋值和 print。
- 如果表达式只含数字和变量, 不含括号优先级, 先用从左到右扫描拿部分分。
- 如果循环次数很小, 甚至可以建语句 AST, 边解析边执行; 但遇到 while 需要回跳时 AST 更稳。
- 函数/作用域不会写时, 先把所有变量当全局, 拿无函数子任务分。

升级方向:

表达式 AST -> 语句 AST -> 全局环境

全局环境 -> 环境栈/局部作用域

解释执行 -> 生成三地址码/字节码

while 执行 -> 状态检测/步数限制

long long 值 -> BigInteger / string / bool / array

最小测试样例:

输入

```

x = 0;
sum = 0;
while (x < 5) {
    x = x + 1;
    sum = sum + x;
}

```

```
print sum;
```

输出

15

补充自测:

输入

```
a = 3;
b = 4;
print -(a + b) * 2;
if (a * b == 12) {
    print 1;
} else {
    print 0;
}
print a / 2;
```

输出

-14

1

1

补充自测 2:

输入

```
i = 3;
while (i != 0) {
    print i;
    i = i - 1;
}
if (!i) print 99; else print 0;
```

输出

3

2

1

99

SIM-06 日期、时间、时区与历法

模块编号: SIM-06

模块名称: 日期转序号、天数差、星期、时区、夏令时与儒略/格里高利历

标签: 模拟、日期、时间、历法、时区、夏令时、儒略历、格里高利历、C++17

一句话用途: 遇到给定日期求相差天数、星期几、加减天、跨时区时间转换、夏令时规则或历史历法切换时, 统一把日期转成整数 day number, 再做加减。

题面触发词:

- 给两个日期, 求相隔多少天。
- 给日期求星期几。
- 给日期加上/减去若干天。
- 给 UTC 时间和时区偏移, 求当地时间。
- 夏令时、生效区间、时钟拨快一小时。
- 儒略历、格里高利历、1582 年 10 月跳日。

什么时候用:

- 日期需要跨月、跨年、跨闰年。
- 需要比较日期先后或求差。
- 题目有时区 offset, 例如 UTC+8、-05:30。
- 题目明确给出夏令时开始/结束日期和偏移规则。
- 历史日期要求区分 Julian / Gregorian。

不要什么时候用：

- 题目只在同一天内算秒数，直接 $h*3600+m*60+s$ 。
- 题目使用农历、节气、天文历法，本模板不覆盖。
- 夏令时按真实国家历史数据库变化，本模板不内置数据库，必须按题面规则模拟。
- 如果题目没有提 1582 切历，不要自行跳过日期，默认 proleptic Gregorian 更稳。

复杂度：

- 日期转 day number: $O(1)$ 。
- day number 转日期: $O(1)$ 。
- 日期差、星期、加减天: $O(1)$ 。
- 处理 n 个日期: $O(n)$ 。

数据范围参考：

- 年份绝对值很大时用 long long。
- 本模板适合普通竞赛日期范围，含公元前需按题面决定是否有 year 0。
- 时区 offset 用分钟保存，避免小数小时。

依赖的标准容器：

- long long: day number、秒数、分钟数。
- string: 解析 YYYY-MM-DD、HH:MM:SS、时区字符串。
- array/vector: 月份天数表，月份按 1-index。

输入如何整理：

```
int y, m, d;
char c1, c2;
cin >> y >> c1 >> m >> c2 >> d; // 2026-05-18
```

接口：

```
days_from_civil(y,m,d) -> Gregorian 日期转 day number, 1970-01-01 为 0。
civil_from_days(z) -> day number 转 Gregorian 日期。
julian_day_number_gregorian(y,m,d) -> Gregorian JDN。
julian_day_number_julian(y,m,d) -> Julian JDN。
diff_days(a,b) = days(b) - days(a)。
weekday = (days + 4) mod 7, 1970-01-01 是 Thursday。
```

最重要思想：日期先转整数

```
日期 y-m-d -> day number
相差天数 = day2 - day1
加 N 天 = civil_from_days(day + N)
星期几 = (day + offset) % 7
```

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Date {
    ll y;
    int m;
    int d;
};

struct DateTime {
    Date date;
    int hh;
    int mm;
    int ss;
};

ll floor_div(ll a, ll b) {
    assert(b > 0);
```

```

    __int128 aa = a, bb = b;
    __int128 q = aa / bb;
    __int128 r = aa % bb;
    if (r < 0) q--;
    assert((__int128)LLONG_MIN <= q && q <= (__int128)LLONG_MAX);
    return (ll)q;
}

bool is_gregorian_leap(ll y) {
    return (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);
}

bool is_julian_leap(ll y) {
    return y % 4 == 0;
}

int month_days_gregorian(ll y, int m) {
    static int md[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (m < 1 || m > 12) throw runtime_error("bad month");
    if (m == 2 && is_gregorian_leap(y)) return 29;
    return md[m];
}

int month_days_julian(ll y, int m) {
    static int md[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (m < 1 || m > 12) throw runtime_error("bad month");
    if (m == 2 && is_julian_leap(y)) return 29;
    return md[m];
}

bool valid_date_gregorian(ll y, int m, int d) {
    if (m < 1 || m > 12) return false;
    return 1 <= d && d <= month_days_gregorian(y, m);
}

bool valid_date_julian(ll y, int m, int d) {
    if (m < 1 || m > 12) return false;
    return 1 <= d && d <= month_days_julian(y, m);
}

// Howard Hinnant 算法: Gregorian, 返回距离 1970-01-01 的天数。
ll days_from_civil(ll y, int m, int d) {
    if (!valid_date_gregorian(y, m, d)) throw runtime_error("bad Gregorian date");
    y -= m <= 2;
    ll era = floor_div(y, 400);
    unsigned yoe = (unsigned)(y - era * 400);
    unsigned mp = (unsigned)(m + (m > 2 ? -3 : 9));
    unsigned doy = (153 * mp + 2) / 5 + (unsigned)d - 1;
    unsigned doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
    return era * 146097 + (ll)doe - 719468;
}

Date civil_from_days(ll z) {
    z += 719468;
    ll era = floor_div(z, 146097);
    unsigned doe = (unsigned)(z - era * 146097);
    unsigned yoe = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
    ll y = (ll)yoe + era * 400;
    unsigned doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
    unsigned mp = (5 * doy + 2) / 153;
    unsigned d = doy - (153 * mp + 2) / 5 + 1;
    unsigned m = mp + (mp < 10 ? 3 : -9);
    y += m <= 2;

```

```

    return {y, (int)m, (int)d};
}

// JDN: 整数儒略日号, 适合比较历史日期。Gregorian 版本。
ll julian_day_number_gregorian(ll y, int m, int d) {
    if (!valid_date_gregorian(y, m, d)) throw runtime_error("bad Gregorian date");
    ll a = (14 - m) / 12;
    ll yy = y + 4800 - a;
    ll mm = m + 12 * a - 3;
    return d + (153 * mm + 2) / 5 + 365 * yy + yy / 4 - yy / 100 + yy / 400 - 32045;
}

// JDN: Julian 历版本。
ll julian_day_number_julian(ll y, int m, int d) {
    if (!valid_date_julian(y, m, d)) throw runtime_error("bad Julian date");
    ll a = (14 - m) / 12;
    ll yy = y + 4800 - a;
    ll mm = m + 12 * a - 3;
    return d + (153 * mm + 2) / 5 + 365 * yy + yy / 4 - 32083;
}

bool is_gregorian_gap_1582(ll y, int m, int d) {
    return y == 1582 && m == 10 && 5 <= d && d <= 14;
}

// 1582 罗马/天主教地区切换示例: 1582-10-15 及之后 Gregorian, 之前 Julian。
ll historical_jdn_1582_switch(ll y, int m, int d) {
    if (is_gregorian_gap_1582(y, m, d)) {
        throw runtime_error("nonexistent date in 1582 switch");
    }
    if (y > 1582 || (y == 1582 && (m > 10 || (m == 10 && d >= 15)))) {
        return julian_day_number_gregorian(y, m, d);
    }
    return julian_day_number_julian(y, m, d);
}

int weekday_1970(Date date) {
    // 0 Sunday, 1 Monday, ..., 6 Saturday. 1970-01-01 is Thursday=4.
    ll z = days_from_civil(date.y, date.m, date.d);
    int w = (int)((z + 4) % 7);
    if (w < 0) w += 7;
    return w;
}

ll datetime_to_utc_seconds(DateTime t, int offset_minutes) {
    ll days = days_from_civil(t.date.y, t.date.m, t.date.d);
    ll local = days * 86400 + t.hh * 3600 + t.mm * 60 + t.ss;
    return local - (ll)offset_minutes * 60;
}

DateTime utc_seconds_to_datetime(ll utc_seconds, int offset_minutes) {
    ll local = utc_seconds + (ll)offset_minutes * 60;
    ll day = floor_div(local, 86400);
    int sec = (int)(local - day * 86400);
    Date date = civil_from_days(day);
    return {date, sec / 3600, sec / 60 % 60, sec % 60};
}

int parse_offset_minutes(const string &s) {
    // 支持 +08:00, -0530, UTC+8, UTC-05:30
    string t = s;
    if (t.rfind("UTC", 0) == 0 || t.rfind("GMT", 0) == 0) {
        t = t.substr(3);
    }
}

```



```

}
if (t.empty()) return 0;
int sign = 1;
int pos = 0;
if (t[pos] == '+') {
    sign = 1;
    pos++;
} else if (t[pos] == '-') {
    sign = -1;
    pos++;
}
string rest = t.substr(pos);
if (rest.empty()) return 0;
int hour = 0, minute = 0;
int colon = (int)rest.find(':');
auto all_digits = [](const string &x) {
    if (x.empty()) return false;
    for (char c : x) {
        if (!isdigit((unsigned char)c)) return false;
    }
    return true;
};
if (colon != -1) {
    string hh = rest.substr(0, colon);
    string mm = rest.substr(colon + 1);
    if (!all_digits(hh) || !all_digits(mm)) throw runtime_error("bad timezone");
    hour = stoi(hh);
    minute = stoi(mm);
} else {
    if (!all_digits(rest)) throw runtime_error("bad timezone");
    if ((int)rest.size() <= 2) {
        hour = stoi(rest);
    } else if ((int)rest.size() == 4) {
        hour = stoi(rest.substr(0, 2));
        minute = stoi(rest.substr(2, 2));
    } else {
        throw runtime_error("bad timezone");
    }
}
if (minute < 0 || minute >= 60) throw runtime_error("bad timezone minute");
return sign * (hour * 60 + minute);
}

void print_date(Date date) {
    cout << date.y << '-';
    cout << setw(2) << setfill('0') << date.m << '-';
    cout << setw(2) << setfill('0') << date.d;
    cout << setfill(' ');
}

void print_datetime(DateTime t) {
    print_date(t.date);
    cout << ' ';
    cout << setw(2) << setfill('0') << t.hh << ':';
    cout << setw(2) << setfill('0') << t.mm << ':';
    cout << setw(2) << setfill('0') << t.ss;
    cout << setfill(' ');
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

```

string op;
cin >> op;

if (op == "diff") {
    Date a, b;
    cin >> a.y >> a.m >> a.d >> b.y >> b.m >> b.d;
    cout << days_from_civil(b.y, b.m, b.d) - days_from_civil(a.y, a.m, a.d) << '\n';
} else if (op == "add") {
    Date a;
    ll delta;
    cin >> a.y >> a.m >> a.d >> delta;
    Date b = civil_from_days(days_from_civil(a.y, a.m, a.d) + delta);
    print_date(b);
    cout << '\n';
} else if (op == "weekday") {
    Date a;
    cin >> a.y >> a.m >> a.d;
    static string name[7] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
    cout << name[weekday_1970(a)] << '\n';
} else if (op == "jdn") {
    string cal;
    Date a;
    cin >> cal >> a.y >> a.m >> a.d;
    if (cal == "gregorian") cout << julian_day_number_gregorian(a.y, a.m, a.d) << '\n';
    else if (cal == "julian") cout << julian_day_number_julian(a.y, a.m, a.d) << '\n';
    else if (is_gregorian_gap_1582(a.y, a.m, a.d)) cout << "INVALID\n";
    else cout << historical_jdn_1582_switch(a.y, a.m, a.d) << '\n';
} else if (op == "tz") {
    DateTime t;
    string from_zone, to_zone;
    cin >> t.date.y >> t.date.m >> t.date.d >> t.hh >> t.mm >> t.ss >> from_zone >> to_zone;
    ll utc = datetime_to_utc_seconds(t, parse_offset_minutes(from_zone));
    DateTime ans = utc_seconds_to_datetime(utc, parse_offset_minutes(to_zone));
    print_datetime(ans);
    cout << '\n';
}

return 0;
}

```

夏令时怎么处理

不要内置真实世界数据库。竞赛题通常会给规则，例如：

每年 3 月第二个星期日 02:00 开始 DST, UTC offset +1h

每年 11 月第一个星期日 02:00 结束 DST

处理策略：

1. 用日期函数算出规则日期。
2. 把开始/结束时刻转成统一秒数。
3. 判断当前时间是否在 [start, end)。
4. 在基础时区 offset 上加 60 分钟。

常用辅助：

```

Date nth_weekday_of_month(ll y, int m, int weekday, int nth) {
    Date first{y, m, 1};
    int w = weekday_1970(first);
    int add = (weekday - w + 7) % 7;
    int day = 1 + add + 7 * (nth - 1);
    return {y, m, day};
}

```

调用示例：

```

11 a = days_from_civil(2024, 2, 28);
11 b = days_from_civil(2024, 3, 1);
cout << b - a << '\n'; // 闰年输出 2

```

常见坑:

- 公历闰年: 能被 400 整除, 或能被 4 整除但不能被 100 整除。
- 1900 不是公历闰年, 2000 是公历闰年。
- 儒略历规则只有 “能被 4 整除就是闰年”。
- 题目没说历史切历时, 默认不要跳过 1582-10-05 到 1582-10-14。
- 1582 切换不同国家时间不一样; 只在题目明确时使用对应规则。
- 时区用分钟, 不要用浮点小时。
- 跨日时本地秒数可能为负, 必须用 floor_div。
- 夏令时的开始/结束瞬间最容易 off-by-one, 统一用半开区间 [start,end)。

暴力/部分替代:

- 日期范围只在同一年: 前缀月份天数即可。
- 不跨闰年: 按普通年模拟。
- 不会 JDN: 对年份不大可从基准日逐日加减, 但要小心效率。
- 时区不跨日期: 只算小时分钟差先拿部分分。
- 夏令时规则复杂: 先忽略 DST, 可能拿非 DST 数据分。

最小测试样例:

输入
diff
2024 2 28 2024 3 1

输出
2

补充自测:

输入
add
2024 2 28 2

输出
2024-03-01

补充自测 2:

输入
weekday
1970 1 1

输出
Thu

补充自测 3:

输入
jdn
historical
1582 10 4

输出
2299160

补充自测 4:

输入
jdn
historical
1582 10 15

输出
2299161

补充自测 5:

输入

tz

2026 5 18 10 0 0 UTC+8 UTC-05:30

输出

2026-05-17 20:30:00

SIM-07 方程求解：高斯消元、模方程、一元方程与多项式

模块编号: SIM-07

模块名称: 方程求解模板: 线性方程组、高斯消元、模意义方程、一元方程迭代、多项式求值与低次公式

标签: 模拟、数学、方程求解、高斯消元、模方程、二分、牛顿法、多项式、C++17、考场模板

一句话用途: 题目把条件写成若干方程、要求求未知数或判无解/多解时, 先判断是线性方程组、模方程、一元连续方程还是低次多项式, 再套对应模板。

题面触发词:

- 给出 n 个未知数和 m 条线性约束, 求每个未知数。
- 解方程组、判断唯一解/无解/无穷多解。
- 所有运算在 $\text{mod } p$ 意义下进行。
- 求 $f(x)=0$ 的一个根, 答案允许误差 $1e-6$ 。
- 给多项式系数, 求某点函数值或求区间内根。
- 一元一次/二次方程, 输出实根。

什么时候用:

- 方程是一次的: 用高斯消元, 实数版或模质数版。
- 单条同余方程 $a \cdot x \equiv b \pmod{m}$: 用 $\text{gcd} + \text{exgcd}$ 。
- 一元连续函数有单调性或已知区间两端异号: 用二分。
- 已有较好初值、函数和导数好算: 可用牛顿法做加速或备用。
- 多项式求值: 用 Horner, 少写幂函数。
- 一元一次/二次: 直接公式, 比迭代更稳。

不要什么时候用:

- 模数不是质数且是多元方程组: 普通模高斯只对可逆主元安全, 复合模需要拆 CRT 或更高阶线性代数。
- 方程里有乘积项如 $x \cdot y$ 、 $x^2 + y^2$: 不是线性方程组, 不能直接高斯。
- 二分区间两端不异号且没有单调性证明: 可能漏根。
- 牛顿法没有好初值或导数会接近 0: 容易飞出有效区间。
- 只要求整数解且变量范围很小: 直接枚举/搜索可能更短。

复杂度:

- 实数高斯消元: $O(m \cdot n \cdot \min(m, n))$, 方阵常记 $O(n^3)$ 。
- 模质数高斯消元: 同上, 每个主元多一次快速幂求逆, 方阵约 $O(n^3 + n \log \text{mod})$ 。
- 单条线性同余: $O(\log \text{mod})$ 。
- 二分求根: $O(\text{iter} \cdot \text{eval})$, 常用 80 到 100 次。
- 牛顿法: $O(\text{iter} \cdot \text{eval})$, 常用 30 到 60 次。
- Horner 多项式求值: $O(\text{deg})$ 。
- 一次/二次公式: $O(1)$ 。

数据范围参考:

- $n \leq 100$: 静态二维数组高斯最舒服。
- $n \leq 500$: $O(n^3)$ 可能还能过, 注意常数和时限。
- 模数为 998244353、 $1e9+7$ 等质数时, 模高斯最稳。
- 浮点答案误差 $1e-6$: 二分 80 次通常足够。
- 多项式次数高且 x 很大时, double 可能溢出, 按题意改 long double 或取模 Horner。

依赖的标准容器:

- 静态数组 $a[\text{MAXN}][\text{MAXN}]$: 方程矩阵, 行列按 1-index。
- double: 实数高斯、二分、牛顿、公式根。
- long long: 模方程系数和答案。

- `vector<double>`: 只在输出低次公式根时临时装答案; 核心矩阵仍是静态数组。

输入如何整理:

```
int m, n;
cin >> m >> n;
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) cin >> a[i][j];
    cin >> a[i][n + 1]; // 右端常数
}
```

接口:

`gauss_real(m,n)` -> 0 无解, 1 唯一解, 2 多解; 答案在 `ans_real[1..n]`。
`gauss_mod_prime(m,n,mod)` -> 模质数方程组, 返回值同上; 答案在 `ans_mod[1..n]`。
`solve_linear_congruence(a,b,mod,x0,step)` -> 解 $a*x \equiv b \pmod{mod}$, 通解 $x=x_0+k*step$ 。
`poly_eval(deg,c,x)` -> Horner 求 $c[0]+c[1]x+\dots+c[deg]x^{deg}$ 。
`poly_derivative_eval(deg,c,x)` -> 多项式导数在 x 的值。
`bisect_poly_root(deg,c,l,r,root)` -> 区间两端异号时二分一个根。
`newton_poly_root(deg,c,start,root)` -> 从 `start` 出发尝试牛顿求根。
`solve_quadratic_formula(a,b,c,roots)` -> 解 $a*x^2+b*x+c=0$, 返回根数, -1 表示任意实数。

先判断题目属于哪一类

题面形式	优先模板	关键检查
$a_1x_1+\dots+a_nx_n=b_1$	实数高斯	EPS、主元、无解/多解
同上但 $\pmod{p}$	模质数高斯	p 是质数、负数取模
$a*x \equiv b \pmod{m}$	<code>gcd + exgcd</code>	<code>gcd(a,m)</code> 是否整除 b
$f(x)=0$, 连续且有区间	二分	两端异号或单调性
$f(x)=0$, 有导数和初值	牛顿	导数别接近 0
多项式代入	Horner	系数顺序和溢出
一次/二次	公式	退化和判别式 EPS

高斯消元的行列约定

增广矩阵按 1-index 存:

`a[i][1..n]` 是第 i 条方程的系数
`a[i][n + 1]` 是右端常数
`ans[1..n]` 是未知数 $x_1..x_n$
`where[col]` 记录第 col 个未知数在哪一行成为主元

判定顺序:

1. 每一列找绝对值最大的主元。
2. 主元行归一化。
3. 消掉其他所有行的这一列。
4. 最后检查 $0 = \text{非零}$, 这是无解。
5. 有变量没主元, 是多解; 所有变量有主元, 是唯一解。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 105;
const double EPS = 1e-10;

double mat_real[MAXN][MAXN];
double ans_real[MAXN];
int where_real[MAXN];

ll mat_mod[MAXN][MAXN];
ll ans_mod[MAXN];
```

```

int where_mod[MAXN];

double coef_poly[MAXN];

int sign_double(double x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}

ll norm_mod(ll x, ll mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

ll mul_mod(ll a, ll b, ll mod) {
    return (ll)((__int128)norm_mod(a, mod) * norm_mod(b, mod) % mod);
}

ll pow_mod(ll a, ll e, ll mod) {
    ll res = 1 % mod;
    a = norm_mod(a, mod);
    while (e > 0) {
        if (e & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return res;
}

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

ll inv_mod_coprime(ll a, ll mod) {
    ll x, y;
    ll g = exgcd(norm_mod(a, mod), mod, x, y);
    if (g != 1) return -1;
    return norm_mod(x, mod);
}

int gauss_real(int m, int n) {
    for (int i = 1; i <= n; i++) where_real[i] = 0;

    int row = 1;
    for (int col = 1; col <= n && row <= m; col++) {
        int sel = row;
        for (int i = row + 1; i <= m; i++) {
            if (fabs(mat_real[i][col]) > fabs(mat_real[sel][col])) {
                sel = i;
            }
        }
        if (fabs(mat_real[sel][col]) < EPS) continue;
    }
}

```

```

    for (int j = col; j <= n + 1; j++) {
        swap(mat_real[sel][j], mat_real[row][j]);
    }

    where_real[col] = row;
    double div = mat_real[row][col];
    for (int j = col; j <= n + 1; j++) mat_real[row][j] /= div;

    for (int i = 1; i <= m; i++) {
        if (i == row) continue;
        double factor = mat_real[i][col];
        if (fabs(factor) < EPS) continue;
        for (int j = col; j <= n + 1; j++) {
            mat_real[i][j] -= factor * mat_real[row][j];
        }
    }
    row++;
}

for (int i = 1; i <= m; i++) {
    bool all_zero = true;
    for (int j = 1; j <= n; j++) {
        if (fabs(mat_real[i][j]) > EPS) {
            all_zero = false;
            break;
        }
    }
    if (all_zero && fabs(mat_real[i][n + 1]) > EPS) return 0;
}

for (int i = 1; i <= n; i++) {
    ans_real[i] = where_real[i] ? mat_real[where_real[i]][n + 1] : 0.0;
}
for (int i = 1; i <= n; i++) {
    if (!where_real[i]) return 2;
}
return 1;
}

int gauss_mod_prime(int m, int n, ll mod) {
    for (int i = 1; i <= n; i++) where_mod[i] = 0;

    int row = 1;
    for (int col = 1; col <= n && row <= m; col++) {
        int sel = 0;
        for (int i = row; i <= m; i++) {
            if (norm_mod(mat_mod[i][col], mod) != 0) {
                sel = i;
                break;
            }
        }
        if (!sel) continue;

        for (int j = col; j <= n + 1; j++) {
            swap(mat_mod[sel][j], mat_mod[row][j]);
            mat_mod[row][j] = norm_mod(mat_mod[row][j], mod);
        }

        where_mod[col] = row;
        ll inv = pow_mod(mat_mod[row][col], mod - 2, mod);
        for (int j = col; j <= n + 1; j++) {
            mat_mod[row][j] = mul_mod(mat_mod[row][j], inv, mod);
        }
    }
}

```

```

    }

    for (int i = 1; i <= m; i++) {
        if (i == row) continue;
        ll factor = norm_mod(mat_mod[i][col], mod);
        if (factor == 0) continue;
        for (int j = col; j <= n + 1; j++) {
            mat_mod[i][j] = norm_mod(mat_mod[i][j] - mul_mod(factor, mat_mod[row][j], mod), mod);
        }
    }
    row++;
}

for (int i = 1; i <= m; i++) {
    bool all_zero = true;
    for (int j = 1; j <= n; j++) {
        if (norm_mod(mat_mod[i][j], mod) != 0) {
            all_zero = false;
            break;
        }
    }
    if (all_zero && norm_mod(mat_mod[i][n + 1], mod) != 0) return 0;
}

for (int i = 1; i <= n; i++) {
    ans_mod[i] = where_mod[i] ? norm_mod(mat_mod[where_mod[i]][n + 1], mod) : 0;
}
for (int i = 1; i <= n; i++) {
    if (!where_mod[i]) return 2;
}
return 1;
}

bool solve_linear_congruence(ll a, ll b, ll mod, ll &x0, ll &step) {
    if (mod <= 0) return false;
    if (mod == 1) {
        x0 = 0;
        step = 1;
        return true;
    }

    ll aa = norm_mod(a, mod);
    ll g = gcd(aa, mod);
    if (b % g != 0) return false;

    ll reduced_mod = mod / g;
    if (reduced_mod == 1) {
        x0 = 0;
        step = 1;
        return true;
    }

    ll inv = inv_mod_coprime(aa / g, reduced_mod);
    if (inv < 0) return false;
    ll rhs = norm_mod(b / g, reduced_mod);
    x0 = mul_mod(rhs, inv, reduced_mod);
    step = reduced_mod;
    return true;
}

double poly_eval(int deg, const double c[], double x) {
    double res = c[deg];
    for (int i = deg - 1; i >= 0; i--) {

```



```

        res = res * x + c[i];
    }
    return res;
}

double poly_derivative_eval(int deg, const double c[], double x) {
    if (deg == 0) return 0.0;
    double res = deg * c[deg];
    for (int i = deg - 1; i >= 1; i--) {
        res = res * x + i * c[i];
    }
    return res;
}

bool bisect_poly_root(int deg, const double c[], double l, double r, double &root) {
    double fl = poly_eval(deg, c, l);
    double fr = poly_eval(deg, c, r);
    if (fabs(fl) < EPS) {
        root = l;
        return true;
    }
    if (fabs(fr) < EPS) {
        root = r;
        return true;
    }
    if (fl * fr > 0) return false;

    for (int it = 1; it <= 100; it++) {
        double mid = (l + r) / 2.0;
        double fm = poly_eval(deg, c, mid);
        if (fabs(fm) < EPS) {
            root = mid;
            return true;
        }
        if (fl * fm <= 0) {
            r = mid;
            fr = fm;
        } else {
            l = mid;
            fl = fm;
        }
        (void)fr;
    }
    root = (l + r) / 2.0;
    return true;
}

bool newton_poly_root(int deg, const double c[], double start, double &root) {
    double x = start;
    for (int it = 1; it <= 60; it++) {
        double fx = poly_eval(deg, c, x);
        double dfx = poly_derivative_eval(deg, c, x);
        if (!isfinite(fx) || !isfinite(dfx) || fabs(dfx) < EPS) return false;
        double nx = x - fx / dfx;
        if (!isfinite(nx)) return false;
        if (fabs(nx - x) < 1e-12) {
            root = nx;
            return fabs(poly_eval(deg, c, root)) < 1e-7;
        }
        x = nx;
    }
    root = x;
    return fabs(poly_eval(deg, c, root)) < 1e-7;
}

```

```

}

int solve_quadratic_formula(double a, double b, double c, double roots[]) {
    if (fabs(a) < EPS) {
        if (fabs(b) < EPS) {
            return fabs(c) < EPS ? -1 : 0;
        }
        roots[1] = -c / b;
        return 1;
    }

    double delta = b * b - 4.0 * a * c;
    if (delta < -EPS) return 0;
    if (fabs(delta) <= EPS) {
        roots[1] = -b / (2.0 * a);
        return 1;
    }

    double sqrt_delta = sqrt(max(0.0, delta));
    double q = -0.5 * (b + (b >= 0 ? sqrt_delta : -sqrt_delta));
    if (fabs(q) < EPS) {
        roots[1] = (-b - sqrt_delta) / (2.0 * a);
        roots[2] = (-b + sqrt_delta) / (2.0 * a);
    } else {
        roots[1] = q / a;
        roots[2] = c / q;
    }
    if (roots[1] > roots[2]) swap(roots[1], roots[2]);
    return 2;
}

void print_real_solution(const string &tag, int n) {
    cout << tag << '\n';
    if (tag == "NO") return;
    for (int i = 1; i <= n; i++) {
        if (i > 1) cout << ' ';
        double x = fabs(ans_real[i]) < 5e-13 ? 0.0 : ans_real[i];
        cout << x;
    }
    cout << '\n';
}

void print_mod_solution(const string &tag, int n) {
    cout << tag << '\n';
    if (tag == "NO") return;
    for (int i = 1; i <= n; i++) {
        if (i > 1) cout << ' ';
        cout << ans_mod[i];
    }
    cout << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cout.setf(ios::fixed);
    cout << setprecision(10);

    string op;
    if (!(cin >> op)) return 0;

    if (op == "real") {

```

```

int m, n;
cin >> m >> n;
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n + 1; j++) cin >> mat_real[i][j];
}
int status = gauss_real(m, n);
if (status == 0) print_real_solution("NO", n);
else if (status == 1) print_real_solution("UNIQUE", n);
else print_real_solution("MANY", n);
} else if (op == "mod") {
    int m, n;
    ll mod;
    cin >> m >> n >> mod;
    if (mod <= 1) {
        cout << "BAD_MOD\n";
        return 0;
    }
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n + 1; j++) {
            cin >> mat_mod[i][j];
            mat_mod[i][j] = norm_mod(mat_mod[i][j], mod);
        }
    }
    int status = gauss_mod_prime(m, n, mod);
    if (status == 0) print_mod_solution("NO", n);
    else if (status == 1) print_mod_solution("UNIQUE", n);
    else print_mod_solution("MANY", n);
} else if (op == "congruence") {
    ll a, b, mod, x0, step;
    cin >> a >> b >> mod;
    if (!solve_linear_congruence(a, b, mod, x0, step)) {
        cout << "NO\n";
    } else {
        cout << x0 << ' ' << step << '\n';
    }
} else if (op == "eval") {
    int deg;
    double x;
    cin >> deg;
    for (int i = 0; i <= deg; i++) cin >> coef_poly[i];
    cin >> x;
    cout << poly_eval(deg, coef_poly, x) << '\n';
} else if (op == "bisect") {
    int deg;
    double l, r, root;
    cin >> deg;
    for (int i = 0; i <= deg; i++) cin >> coef_poly[i];
    cin >> l >> r;
    if (bisect_poly_root(deg, coef_poly, l, r, root)) {
        cout << root << ' ' << poly_eval(deg, coef_poly, root) << '\n';
    } else {
        cout << "NO_BRACKET\n";
    }
} else if (op == "newton") {
    int deg;
    double start, root;
    cin >> deg;
    for (int i = 0; i <= deg; i++) cin >> coef_poly[i];
    cin >> start;
    if (newton_poly_root(deg, coef_poly, start, root)) {
        cout << root << ' ' << poly_eval(deg, coef_poly, root) << '\n';
    } else {
        cout << "FAIL\n";
    }
}

```

```

    }
} else if (op == "quad") {
    double a, b, c;
    double roots[3] = {0, 0, 0};
    cin >> a >> b >> c;
    int cnt = solve_quadratic_formula(a, b, c, roots);
    if (cnt < 0) {
        cout << "INF\n";
    } else {
        cout << cnt << '\n';
        for (int i = 1; i <= cnt; i++) {
            double x = fabs(roots[i]) < 5e-13 ? 0.0 : roots[i];
            cout << x << '\n';
        }
    }
}

return 0;
}

```

调用示例:

```

// 2x + y = 5
// x - y = 1
// 高斯后得到 x=2, y=1。
int m = 2, n = 2;
mat_real[1][1] = 2; mat_real[1][2] = 1; mat_real[1][3] = 5;
mat_real[2][1] = 1; mat_real[2][2] = -1; mat_real[2][3] = 1;
int status = gauss_real(m, n);
if (status == 1) {
    cout << ans_real[1] << ' ' << ans_real[2] << '\n';
}

// 多项式 1 - 3x + 2x^2 在 x=3 的值。
double c[3];
c[0] = 1;
c[1] = -3;
c[2] = 2;
cout << poly_eval(2, c, 3) << '\n'; // 10

```

二分和牛顿怎么选

二分是“慢但稳”:

- 必须能保证根在区间里。
- 最常见保证是 $f(l)$ 与 $f(r)$ 异号。
- 如果函数单调, 也可以二分找 $f(x) \geq 0$ 的边界。

牛顿是“快但挑初值”:

$$x_{k+1} = x_k - f(x_k) / f'(x_k)$$

- 初值离根太远可能发散。
- 导数接近 0 时要停。
- 考场建议: 能二分先二分; 牛顿只在题目明确或需要加速时用。

多项式求根的低心智路线

一次: $a \cdot x + b = 0$, 直接 $-b/a$ 。

二次: $a \cdot x^2 + b \cdot x + c = 0$, 判别式 $\Delta = b^2 - 4ac$ 。

三次及以上: 如果只求某个区间一个根, 用二分/牛顿; 不要临场硬抄三次公式。

如果题目要求“所有实根”:

- 二次公式可以全部输出。
- 高次多项式需要导数分段、Sturm、或题面给出特殊性质; 普通 SIM 模板不强行覆盖。
- 只给误差要求且区间小, 可以按题意扫描小区间, 再对异号段二分, 但偶重根可能不会异号。

常见坑:

- 高斯消元一定要先判无解, 再判多解。
- 实数高斯主元要选绝对值最大行, 别直接拿当前行。
- EPS 不是越小越好; 普通 double 用 $1e-9$ 到 $1e-12$ 。
- 多解时自由变量可以先置 0, 但题目如果要求参数形式, 要单独输出自由变量。
- 模高斯这里默认 mod 是质数; 复合模下非零数不一定有逆元。
- 读入负数系数后要 $((x \% \text{mod}) + \text{mod}) \% \text{mod}$ 。
- $a \cdot x \equiv b \pmod{m}$ 有解条件是 $\gcd(a, m) \mid b$ 。
- 二分根要求连续函数; 离散答案二分和连续二分不是同一件事。
- 区间两端同号不代表没有根, 例如 $(x-1)^2=0$ 。
- 牛顿法要防导数为 0、结果变成 nan/inf。
- Horner 系数顺序必须统一, 本模板用低次到高次: $c[0], c[1], \dots, c[\text{deg}]$ 。
- 二次公式里 delta 接近 0 时按一个根处理, 避免输出两个几乎相同的根。
- 输出浮点答案记得 `fixed << setprecision(...)`。

暴力/部分替代:

- 未知数 $n \leq 3$ 且整数范围小: 枚举所有变量检查方程。
- 实数方程组只要求小数据: 可以用 Cramer's rule 解 $2 \times 2 / 3 \times 3$, 但高斯更通用。
- 模方程变量很少且模数小: 直接枚举 $0 \dots \text{mod}-1$ 。
- 一元方程区间很短且答案是整数: 枚举整数点。
- 多项式次数低: 优先一次/二次公式。
- Newton 不稳时, 退回二分。

升级方向:

整数小范围枚举 -> 实数/模高斯

单条同余 -> exgcd

模质数方程组 -> 模高斯

连续一元方程 -> 二分

二分太慢且导数好算 -> 牛顿

二次以内 -> 公式

高次所有根 -> 导数分段 / Sturm / 专题数学

最小测试样例:

输入

```
real
2 2
2 1 5
1 -1 1
```

输出

```
UNIQUE
2.0000000000 1.0000000000
```

补充自测 1:

输入

```
mod
2 2 1000000007
2 1 5
1 -1 1
```

输出

```
UNIQUE
2 1
```

补充自测 2:

输入

```
congruence
14 30 100
```

输出

```
45 50
```

补充自测 3:

输入

eval

2

1 -3 2

3

输出

10.0000000000

补充自测 4:

输入

bisect

2

-2 0 1

0 2

输出示例

1.4142135624 0.0000000000

补充自测 5:

输入

quad

1 -3 2

输出

2

1.0000000000

2.0000000000